REVIEWED DESIGN PROPOSAL v2.0

Invoice Desktop - detailed UML and system design

Windows desktop | Python + PySide6 | SQLite + SQLAlchemy | ReportLab + docxtpl

A design for independent screens, testable invoice rules and versioned document templates.

Design commitment	Meaning
What remains confirmed	Sales invoices for Indian goods and services; automatic totals and amount in words; numeric-only numbering reset each financial year; individual and inclusive date-range viewing/export/sharing; PDF and Word output; purchase capture later.
Frontend / backend boundary	Qt screens communicate through typed inbound interfaces and plain request/result objects. The backend contains use cases and domain rules; it can execute without Qt. Repositories and document libraries are injected adapters.
Template boundary	Financial facts, template versions, branding snapshots, preview bindings and generated artifacts are separate objects. Changing appearance cannot change tax calculations or overwrite issued documents.
Practical deployment	One owner and one main Windows workstation; offline core; a local database; background jobs; manual email/WhatsApp handoff; encrypted complete backups. Supporting features remain the proposed baseline.
Status of this deliverable	Architecture documentation, not an implemented or tested application. PlantUML syntax and document rendering are checked. Runtime SOLID conformance, accounting correctness and production readiness require the specified implementation tests.

WHAT IS IN THIS REVISION

28 UML views; a candid audit of the earlier version; frontend/backend contracts; template lifecycle and document binding; class, package, sequence, state, activity and deployment diagrams; import rules; interface guarantees; requirements traceability; acceptance scenarios; and implementation decisions.

START HERE

Read the audit on page 2, then views 02-05 for coupling and API contracts, views 09-11 and 15-16 for template design, and the SOLID/enforcement tables after the diagrams. The editable pack includes .puml source, SVG diagrams and an offline gallery.

Audit of the earlier UML

The first version was directionally sound; it was not sufficient as a detailed implementation contract for templates and frontend/backend separation.

Area	Decision	Review finding and correction
Layering and offline deployment	Keep	The previous design already separated presentation, application, domain and adapters, and correctly kept SQLite local. This revision makes allowed imports and inbound capabilities explicit.
Template support	Expand substantially	The earlier model referenced template versions but did not define template catalogue, draft validation, publishing, activation, retirement, branding snapshots or the actual preview workflow. Views 09-11, 15-16, 20 and 23 now cover these.
Frontend-facing boundary	Make explicit	Views previously called use cases without a detailed public contract. Views 03-05 now define input ports, immutable DTOs, version/conflict handling and the Qt task bridge.
Persistence interface segregation	Refine	The former IIssuanceRecordWriter grouped snapshot, audit and job writes. View 12 splits these into narrow ports coordinated by a transaction scope.
Immutable invoice lines	Clarify model	A snapshot reference to InvoiceLine could be read as a reference to mutable data. View 06 uses distinct FrozenLine value objects and separate financial snapshots.
Rendering and progress coupling	Remove ambiguity	The earlier sequence views mixed backend work with UI notification shorthand. Views 14, 16 and 17 now show queued presentation calls or public job-status queries; backend workers have no widget reference.
Artifact identity	Strengthen	Snapshot + template alone did not fully specify branding/font changes. RenderBinding pins content, template, branding, locale, schema and renderer versions; artifact keys derive from that binding.
SOLID assurance	Add enforcement plan	A diagram does not prove code follows SOLID. This revision specifies forbidden imports, adapter contract tests, headless backend tests and failure scenarios that will check the implementation.

READING GUIDE

How to read the model

Different UML views describe different concerns; no single diagram is expected to contain every class and operation.

Views	Concern	What to look for
01-05	System and interface boundary	Actors, module dependency direction, passive Qt views, invoice APIs, template/document/export/job APIs.
06-08	Business model	Invoice identity, frozen financial facts, calculation responsibilities, masters, payments, adjustments and audit.
09-12	Templates and implementation ports	Template family/draft/version, branding and bindings, neutral document model, renderer adapters and atomic persistence ports.
13-18	Runtime collaboration	Issue once, UI request/result interaction, publish a template, preview a draft, recover document generation and export a date range.
19-24	Lifecycles and data relationships	Invoice/template/job states and the logical SQLite schemas for invoice, template, document and operational records.
25-28	Operations and expansion	Complete backups, safe restore, Windows deployment and independent future purchase capture.

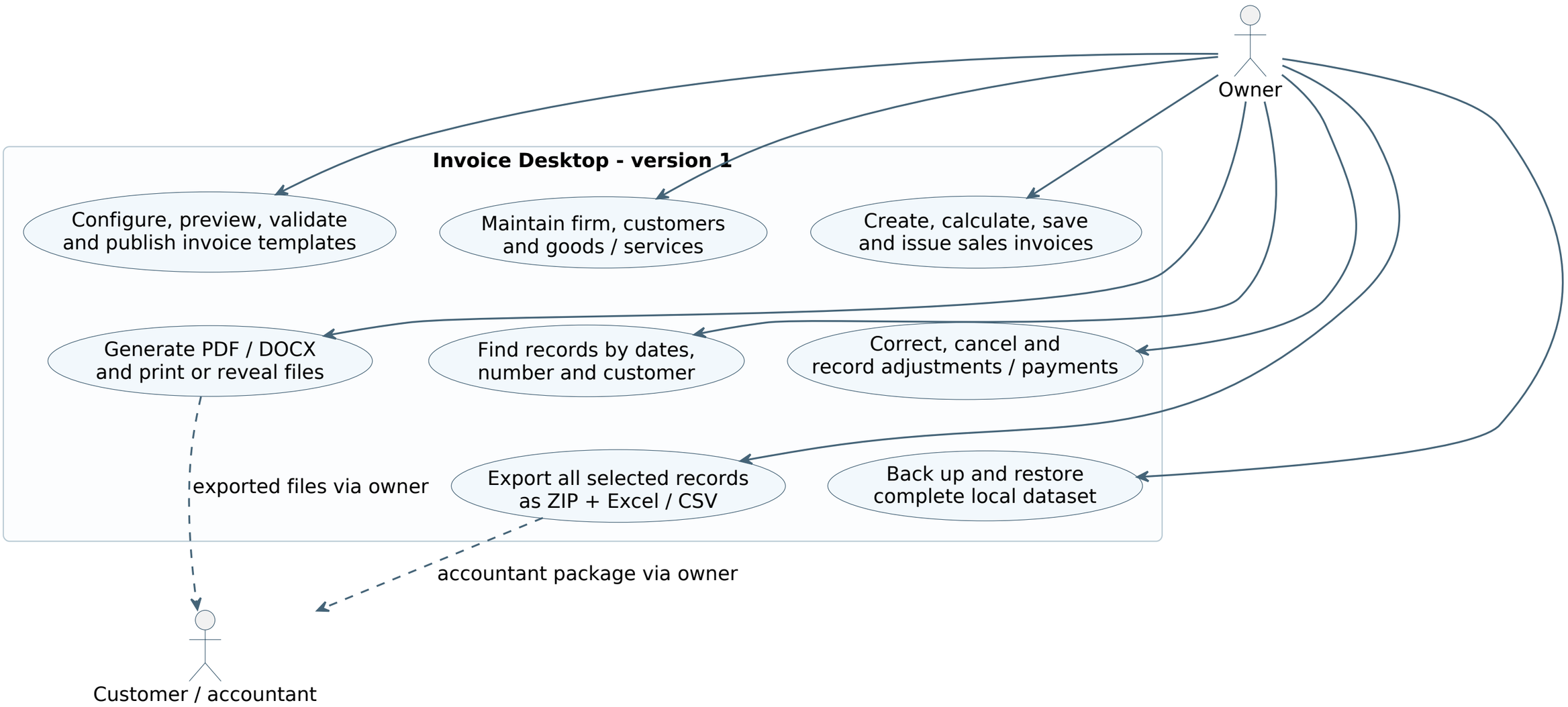
TERMS USED THROUGHOUT

Term	Meaning
Port	A small Python interface/Protocol owned by the application or domain. It describes a capability without selecting a library.
Adapter	A concrete implementation: SQLAlchemy repository, ReportLab renderer, docxtpl renderer, local file store or Windows handoff.
DTO	An immutable data-transfer object crossing the UI boundary. It contains values and identifiers, not live domain entities or database sessions.
Snapshot / binding	A snapshot freezes invoice facts. A render binding freezes which template, branding and rendering versions present those facts.
Composition root	Startup code that creates handlers/adapters and injects the relevant interfaces into each view model. It contains wiring, not invoice rules.

USE-CASE DIAGRAM

01 Owner workflows and system scope

The owner operates the offline desktop app; external recipients receive exported files and have no direct database access.



01

Confirmed scope: sales invoices for goods and services; numeric-only annual numbering; automatic totals and words; PDF/DOCX; inclusive date-range view and sharing. Supporting master, payment and backup features remain the proposed baseline.

02

Template configuration is now a first-class workflow. The initial release uses approved A4 PDF/DOCX layout pairs with editable branding and optional content; arbitrary executable templates and a full canvas designer are outside version 1.

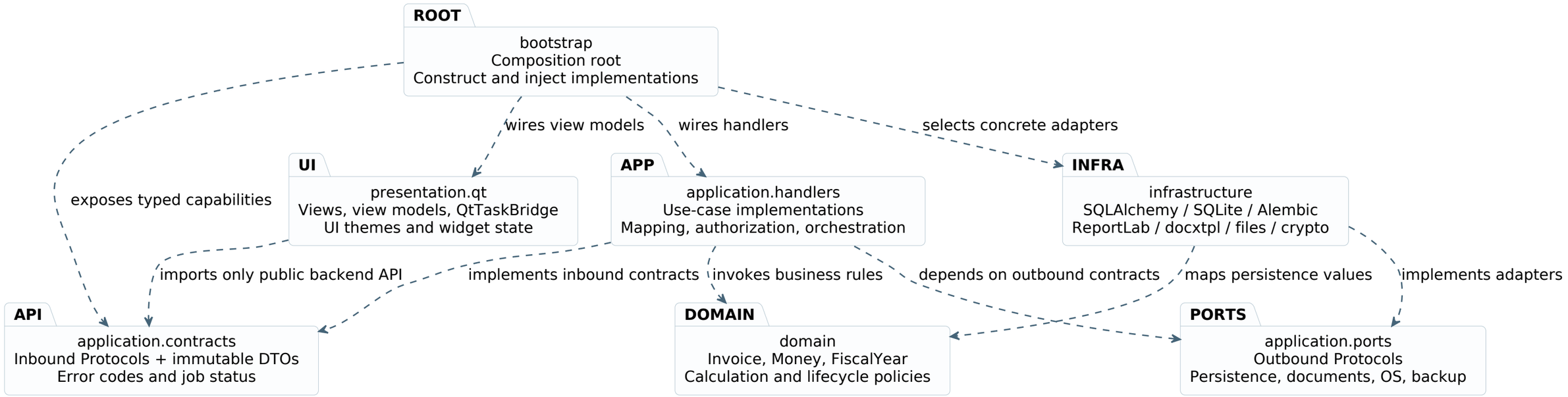
03

Sharing prepares files and reveals their location. The owner attaches them in email or WhatsApp. Purchase capture, automatic message delivery, cloud links and multi-computer access are future modules.

PACKAGE DIAGRAM

02 Frontend, backend and dependency direction

The desktop frontend and backend are separate Python packages, connected by explicit contracts rather than concrete implementation imports.



01

The UI imports application.contracts, not handlers, domain aggregates, SQLAlchemy models or renderer libraries. DTOs contain plain IDs, dates, integer paise, tuples and status/error codes; no QObject, QWidget, Session or library-specific objects cross the boundary.

02

Application handlers and domain rules never import presentation. Domain and contract packages use the standard library only. Only bootstrap knows concrete frontend, handlers and infrastructure adapters; no global service locator is exposed to views.

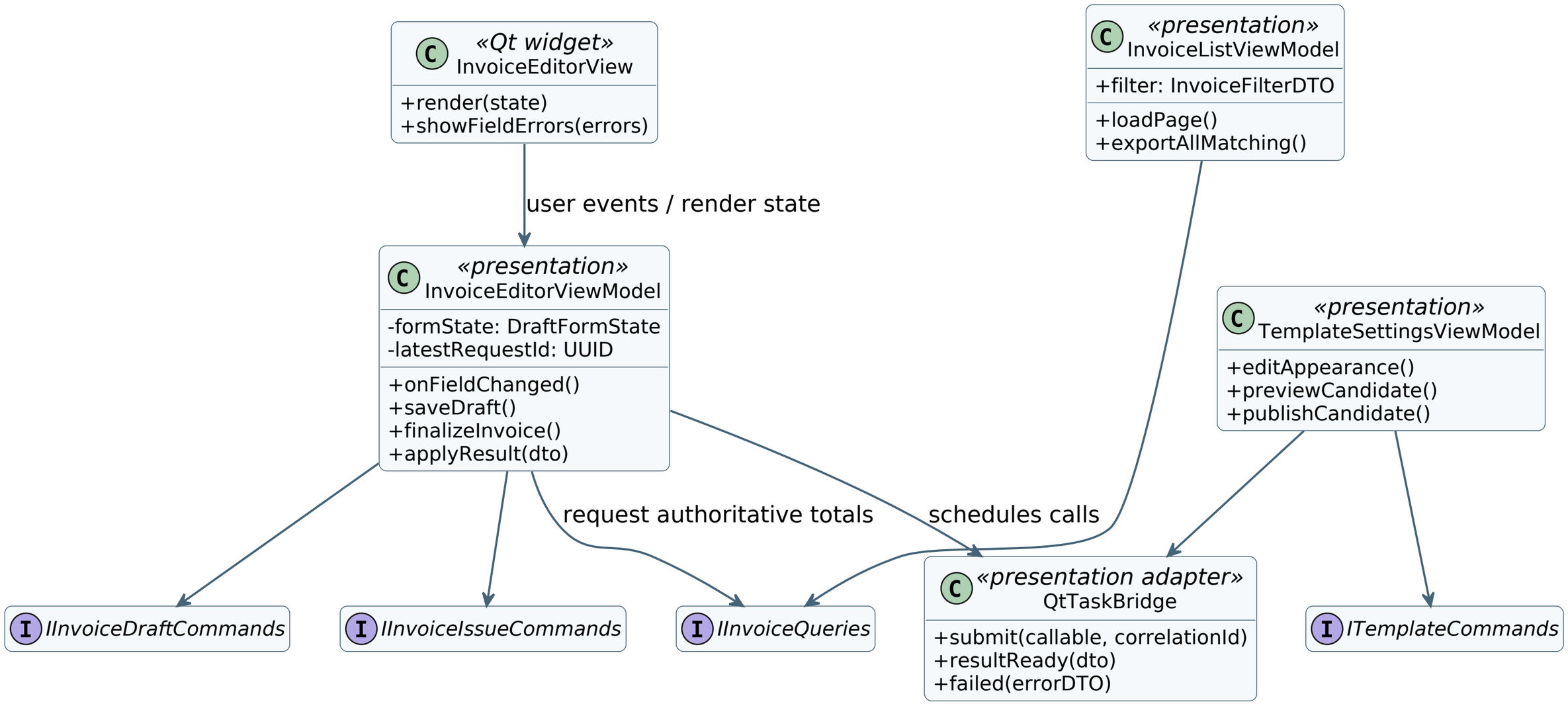
03

This is an in-process modular monolith, not a network API. Logical separation is sufficient for version 1. A future HTTP/IPC adapter may implement the same public contracts, but needs explicit serialization, authentication and failure semantics.

CLASS DIAGRAM

03 Passive views, view models and asynchronous calls

Widgets manage display and input. View models translate user actions into backend requests and map result DTOs to UI state.



01

A view model copies form values into an immutable request before dispatch. QtTaskBridge owns QThreadPool integration and queues results back to the GUI thread. Each backend operation creates its own unit of work; no database connection is shared through a widget.

02

Client-side checks improve interaction, while backend validation remains authoritative. Live totals come from IInvoiceQueries.calculateDraft; the frontend does not duplicate tax or amount-in-words formulas. Stale asynchronous results are ignored by request ID.

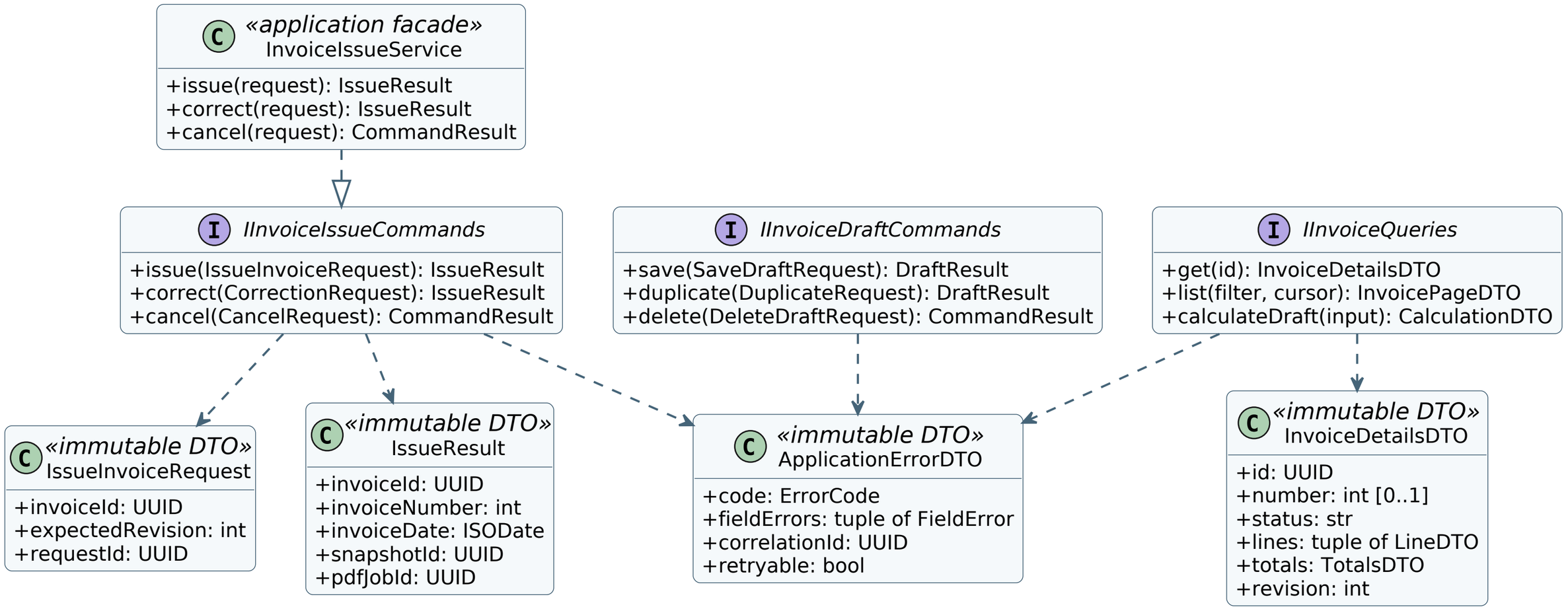
03

The list view uses paged queries. Export receives the filter itself, not the visible row collection. TemplateSettingsViewModel also receives template queries and preview ports; the diagram shows its command dependency to keep this view readable.

CLASS DIAGRAM

04 Public invoice contracts and plain DTOs

Inbound ports are the stable frontend/backend boundary. Implementations can be called by Qt, a headless test or a later transport adapter.



01

Use frozen dataclasses and typing.Protocol for these contracts. Signatures abbreviate Result[T, ApplicationErrorDTO]; every method has typed success or failure. DTOs are snapshots of data, never mutable domain or ORM objects.

02

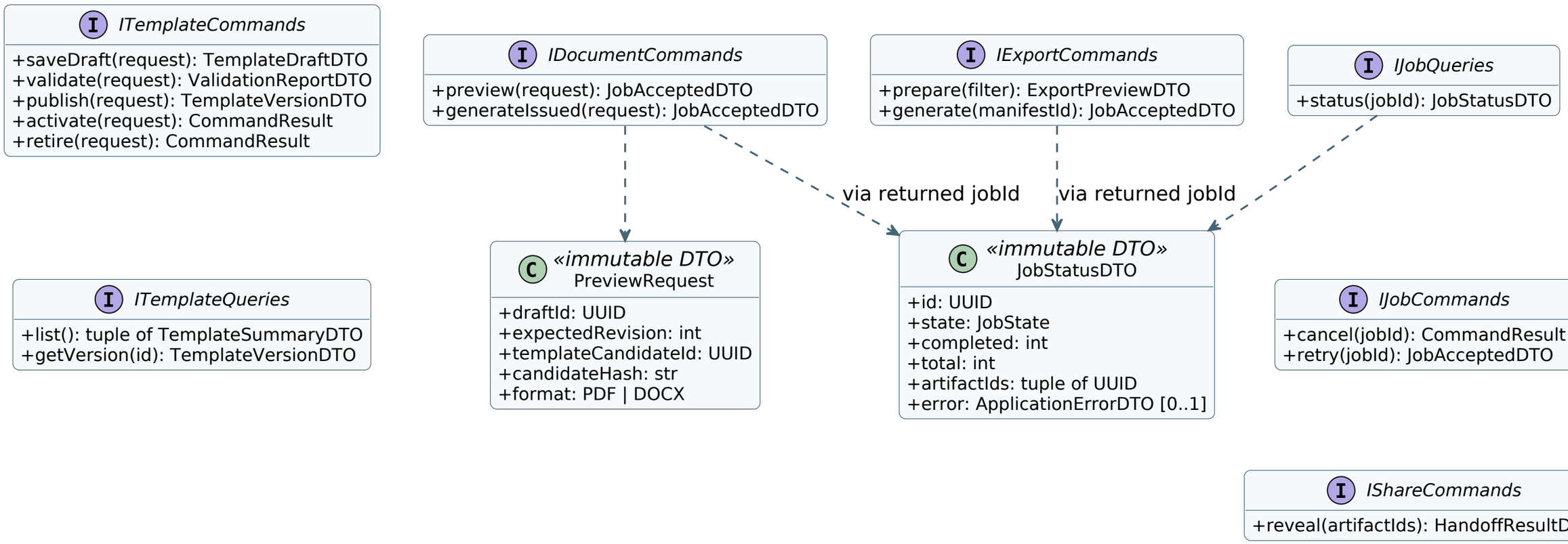
InvoiceIssueService implements all three port methods and delegates to separate IssueInvoiceHandler, CorrectInvoiceHandler and CancelInvoiceHandler objects. A request ID plus payload hash makes issuance idempotent; reusing a key with different input is a conflict. expectedRevision detects stale edits. Invoice number is null for drafts and a positive integer after issuance; the internal UUID remains the durable identity across financial years.

03

Errors include VALIDATION, CONFLICT, NOT_FOUND, UNSUPPORTED_TAX_TREATMENT, TEMPLATE_UNAVAILABLE, STORAGE_FAILURE and RECOVERY_REQUIRED. The UI translates codes to messages; backend exceptions and SQL traces remain in local diagnostics.

05 Template, preview, export and job contracts

The frontend requests capabilities. It does not choose renderer classes, pass database sessions or read internal artifact paths.



01

Additional cohesive ports cover IMasterDataCommands/Queries, IPaymentCommands/Queries and IBackupCommands/Queries. Capability groups are injected separately; a screen receives only the capabilities it uses.

02

Preview requests bind both invoice revision and template candidate hash. A later edit cannot quietly change the content of an already-running preview. UI polling of IJobQueries or QtTaskBridge signals uses plain JobStatusDTO objects.

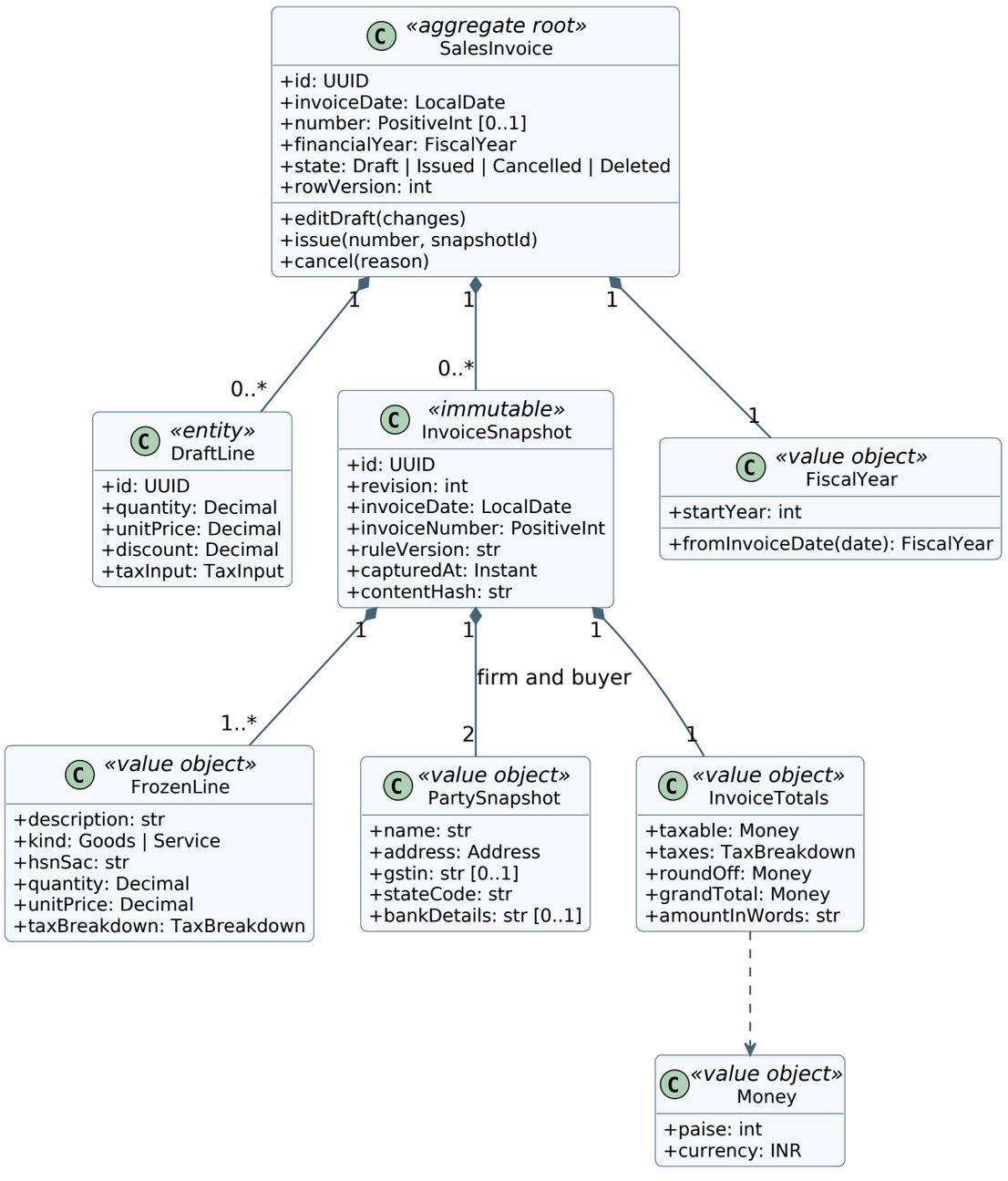
03

Artifact IDs are resolved by backend services. Export/save/open operations validate selected destinations. Windows handoff does not mean sent or delivered; direct email or messaging would require a separate delivery contract.

CLASS DIAGRAM

06 Invoice identity and immutable financial facts

The sales aggregate contains financial facts and lifecycle rules. It has no knowledge of Qt widgets, DOCX files or PDF layout.



01

Drafts can be empty; issuance requires at least one valid line. The displayed invoice number is only 1, 2, 3, etc. Financial year is internal metadata, derived from invoice date, with separate sequences beginning at 1.

02

FrozenLine is a distinct value object, not a pointer back to mutable draft or catalogue data. Issued party, line and total snapshots survive later changes to customer details, prices or tax settings.

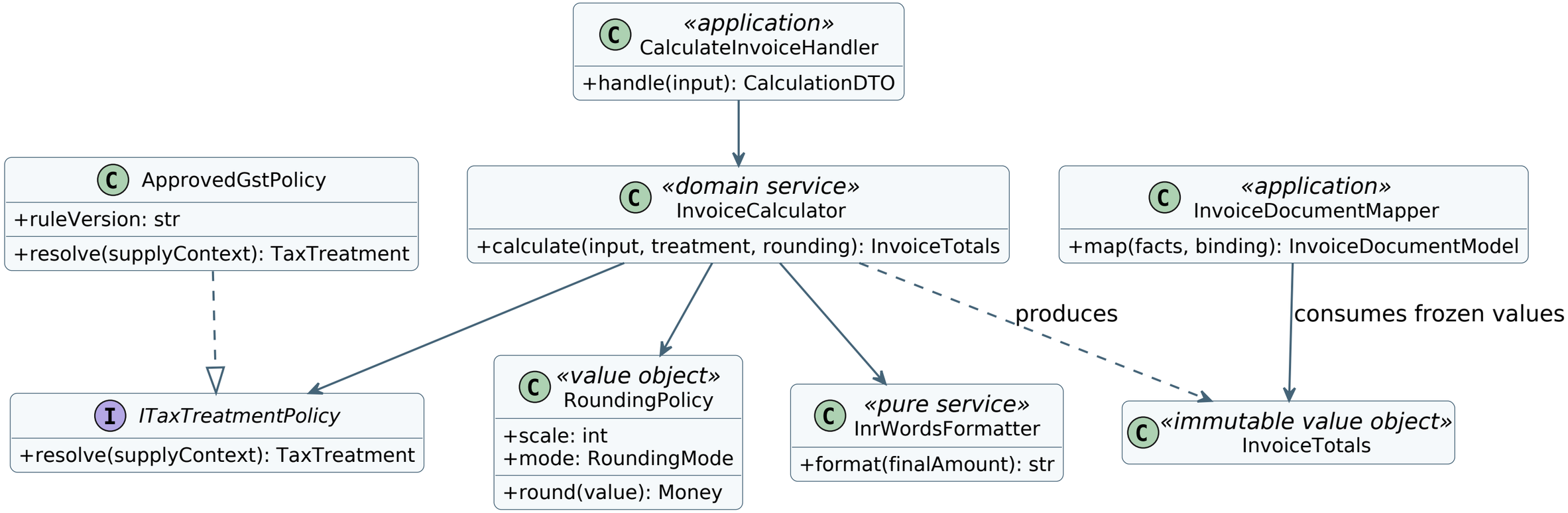
03

Document appearance is pinned by a separate RenderBinding (view 10). SalesInvoice and InvoiceSnapshot do not depend on TemplateVersion, ReportLab or docxtpl. Snapshot values include approved terms and payment details; templates cannot override financial facts.

CLASS DIAGRAM

07 Calculation policy and formatting responsibilities

There is one authoritative calculation path. The UI and all document formats consume its results.



01

InvoiceDocumentMapper consumes frozen financial facts. It has no dependency on InvoiceCalculator and cannot invoke tax calculations. Sharing a value-object schema does not couple document layout to the calculation implementation.

02

Use Decimal for quantity, prices, discounts, rates and intermediate values; integer paise for settled amounts. Configure and test rounding explicitly. Convert the final rounded amount, including paise, to words.

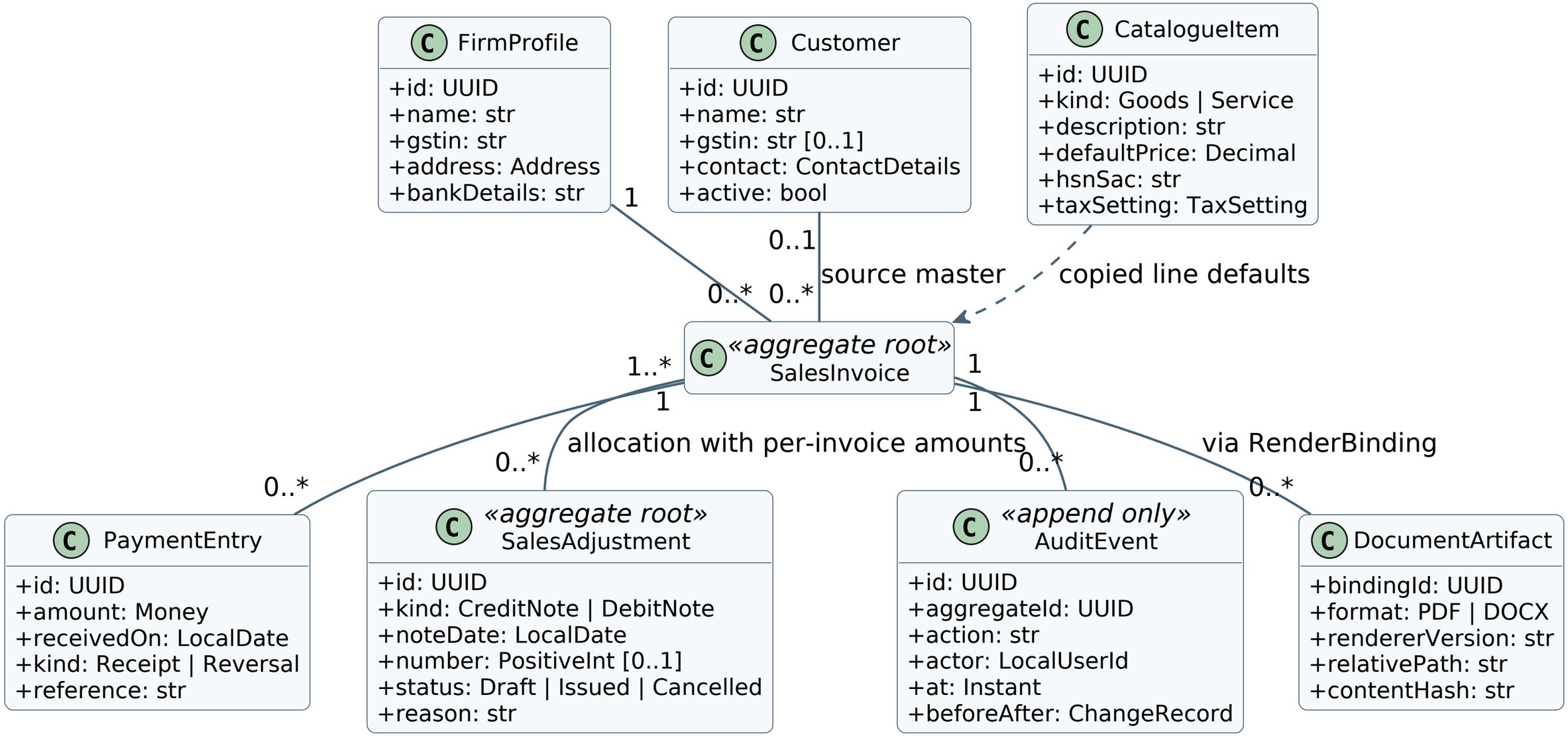
03

Tax policy uses explicit supply context and approved treatment. It must not infer every service or special transaction solely from two state codes. Unsupported treatments return validation errors; a template never chooses a tax rate.

CLASS DIAGRAM

08 Masters, payments, adjustments and audit history

Reusable masters speed entry. Payments, tax adjustments, document files and audit history remain separate concepts.



01

Payment status is derived from receipts, reversals and allocated adjustments; it is not the invoice lifecycle. A paid invoice does not disappear from the date-range sales register.

02

Credit/debit notes are separate issued documents with their own numeric sequence per financial year and document kind. They may reference older invoices and are selected by note date in the adjustment register.

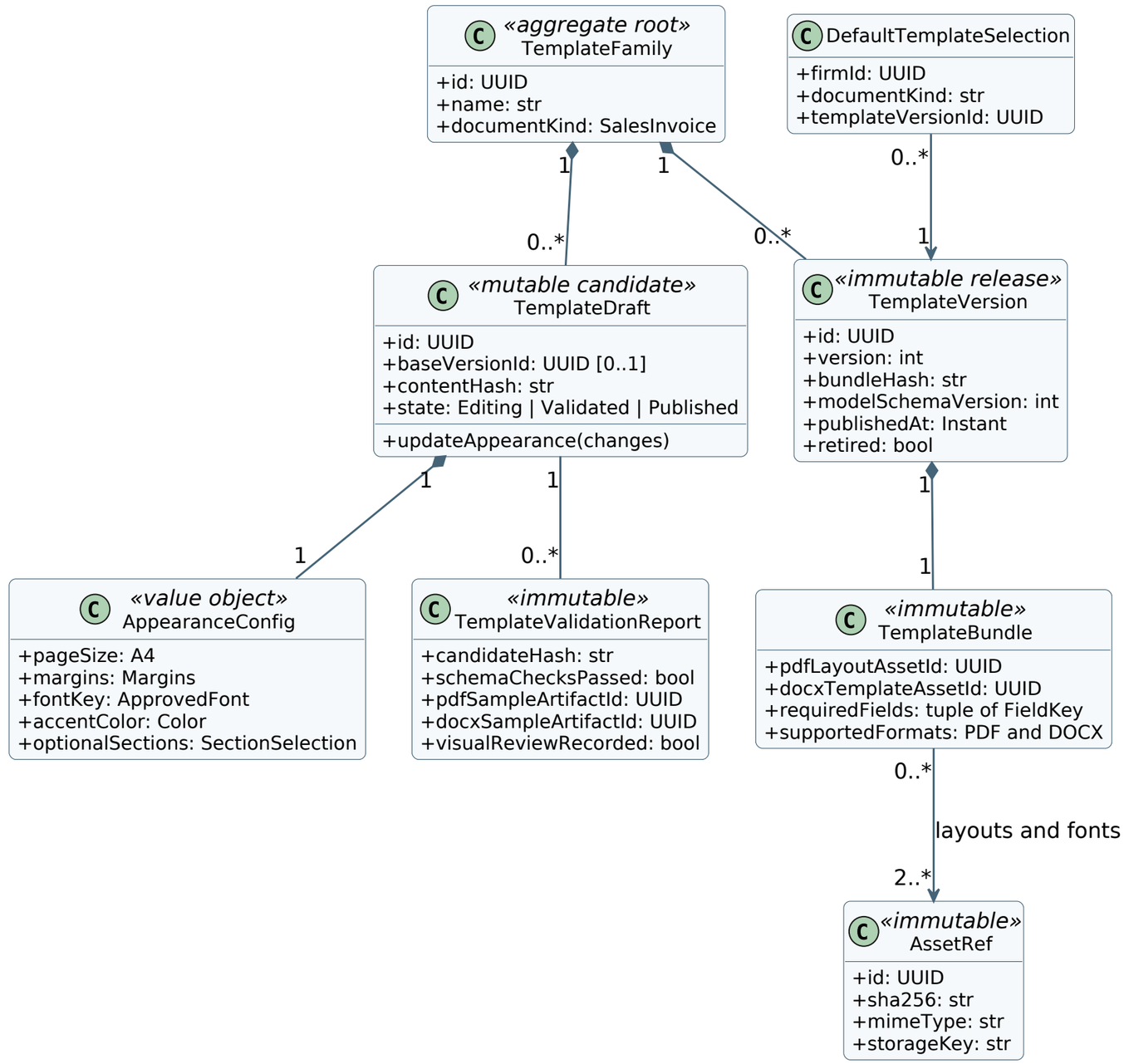
03

Audit history is append-only through application contracts. This is an audit trail, not a claim of tamper-proof storage against a Windows administrator.

CLASS DIAGRAM

09 Template catalogue, revisions and appearance

Template editing is independent of invoice calculation. Publishing produces a versioned PDF/DOCX bundle with validated appearance settings.



01

Version 1 offers approved layouts and constrained appearance settings. PDF layout descriptors and DOCX files are paired assets. Application UI themes and invoice appearance are separate settings; a dark application theme does not recolour printed invoices.

02

Validation is tied to the exact candidate hash. Any edit invalidates the old report. Publishing requires required-field checks, both format samples and recorded visual review; python-docx/docxtpl generates DOCX but does not itself verify Word pagination.

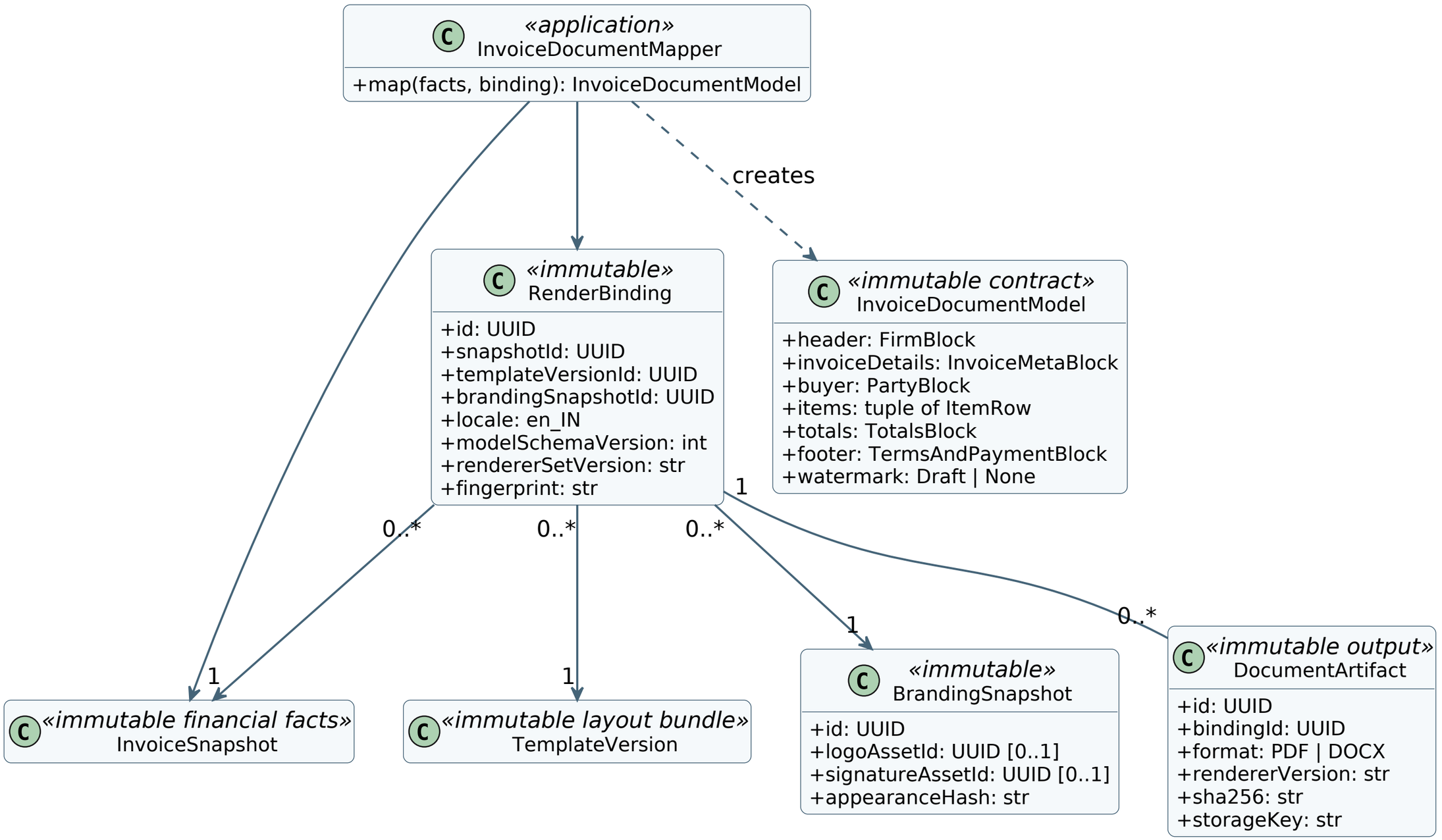
03

TemplateVersion content is immutable after publication; retirement is availability metadata only. Existing bindings may still use retired versions. Changing the default affects new bindings and never rewrites historical documents.

CLASS DIAGRAM

10 Financial facts, template binding and document model

A binding freezes which layout, branding and rendering versions are used for a particular financial snapshot.



01

The binding fingerprint covers snapshot content, template bundle, branding assets, locale, document-model schema and renderer-set version. Artifact uniqueness uses binding ID + format + concrete renderer version, so a logo or template change cannot overwrite an old file.

02

Header, buyer, invoice metadata, item rows, totals and footer are explicit template sections. Invoice number, GST amounts and amount in words are supplied values. The mapper formats these values; it does not calculate or alter them.

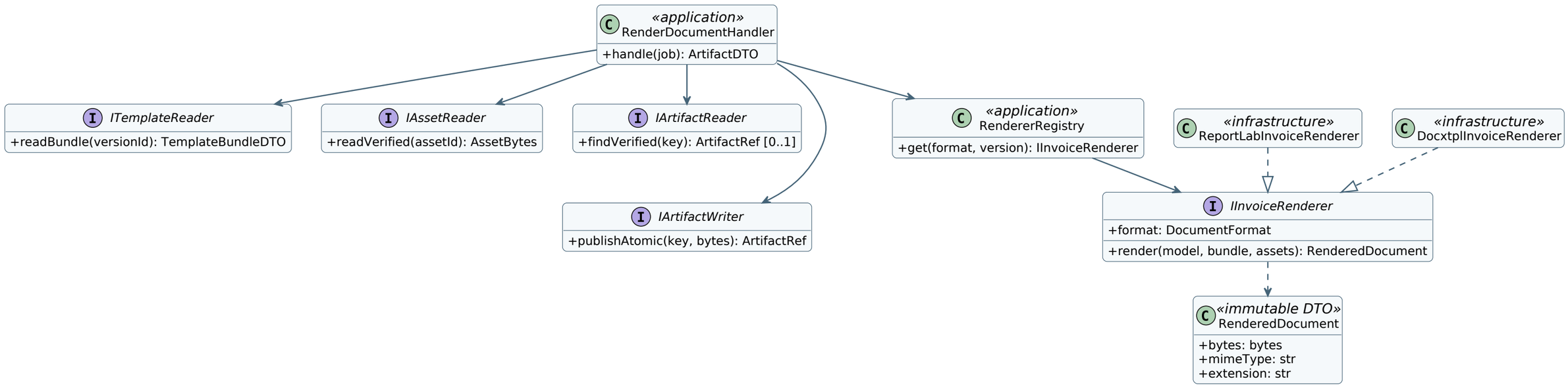
03

Preview uses a distinct PreviewBinding with captured draft facts, draft revision and candidate hash; it has no issued number and always carries a DRAFT watermark. A new appearance for an issued invoice creates a labelled derived copy rather than replacing the canonical issued artifact.

CLASS DIAGRAM

11 PDF and Word adapters behind one render contract

Both adapters accept the same neutral document model. Only the adapters import ReportLab, docxtpl or python-docx.



01

PDF uses approved ReportLab layouts; DOCX uses approved .docx placeholders and item-row loops through docxtpl. Use strict missing-field validation and XML escaping. Approved tags substitute fields, repeat rows and select optional blocks; templates do not contain financial formulas. Arbitrary executable uploads are outside the initial release.

02

Renderers preserve data and return bytes plus format metadata or a typed rendering error. They never allocate invoice numbers, write business records, show dialogs or perform network calls. Font, logo and signature assets are resolved before rendering.

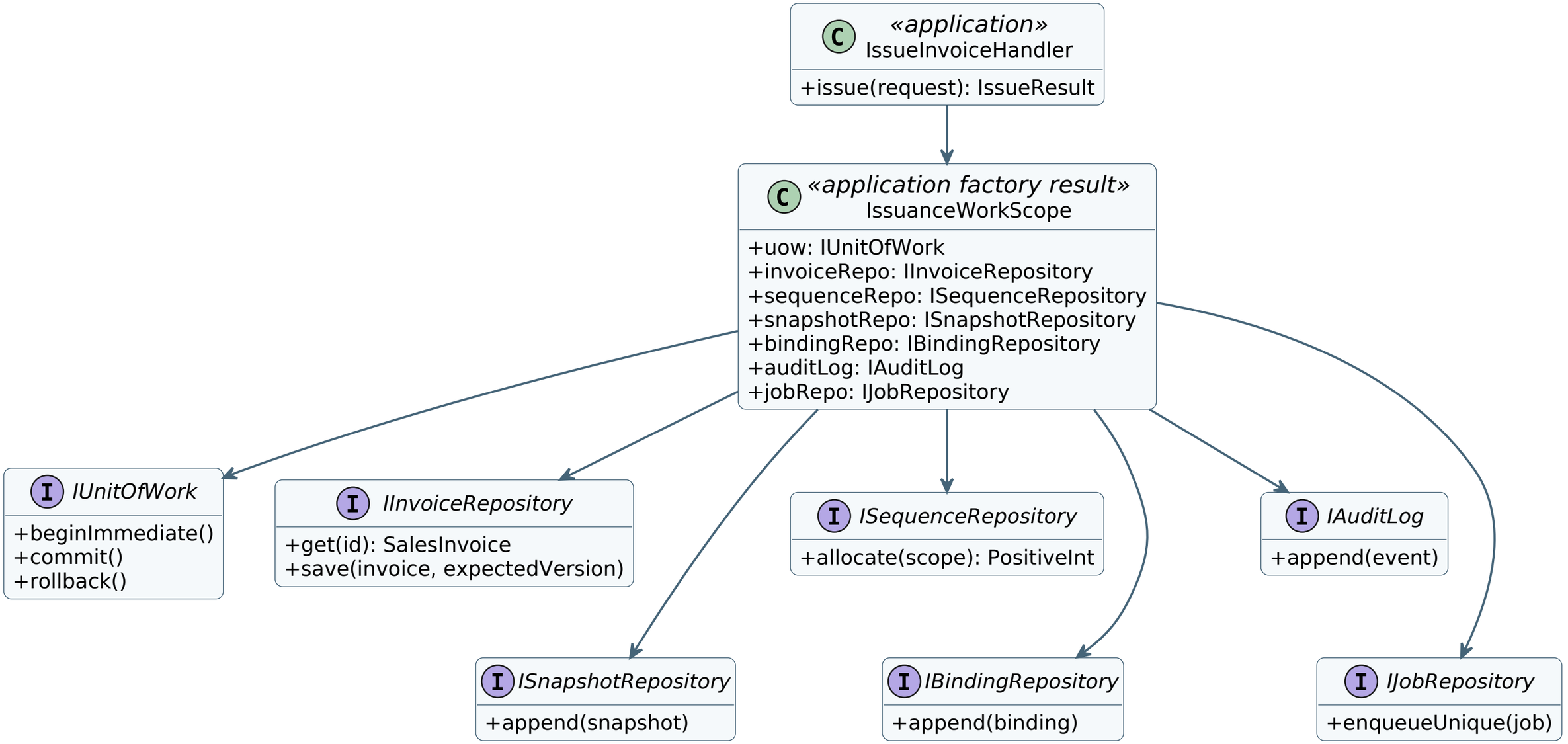
03

PDF and Word must agree on facts, totals and branding. They are not promised identical pagination or pixels. The canonical PDF is archived; historical layout reproducibility requires retained assets and renderer versions, not only a template name.

CLASS DIAGRAM

12 Atomic issuance using segregated persistence ports

A work-scope factory supplies narrow repository interfaces participating in one transaction; repositories do not commit independently.



01

This replaces the earlier IIssuanceRecordWriter grouping. Snapshot storage, audit append and job enqueue have separate interfaces. IssuanceWorkScope is an immutable dependency bundle, not a service locator or a class containing business logic.

02

SQLAlchemyUnitOfWork and SQLAlchemy repositories are infrastructure adapters over SQLite. Every repository in a scope uses the same session/connection. Verify emitted transaction SQL actually takes the intended SQLite write lock; ORM defaults alone are not the numbering guarantee.

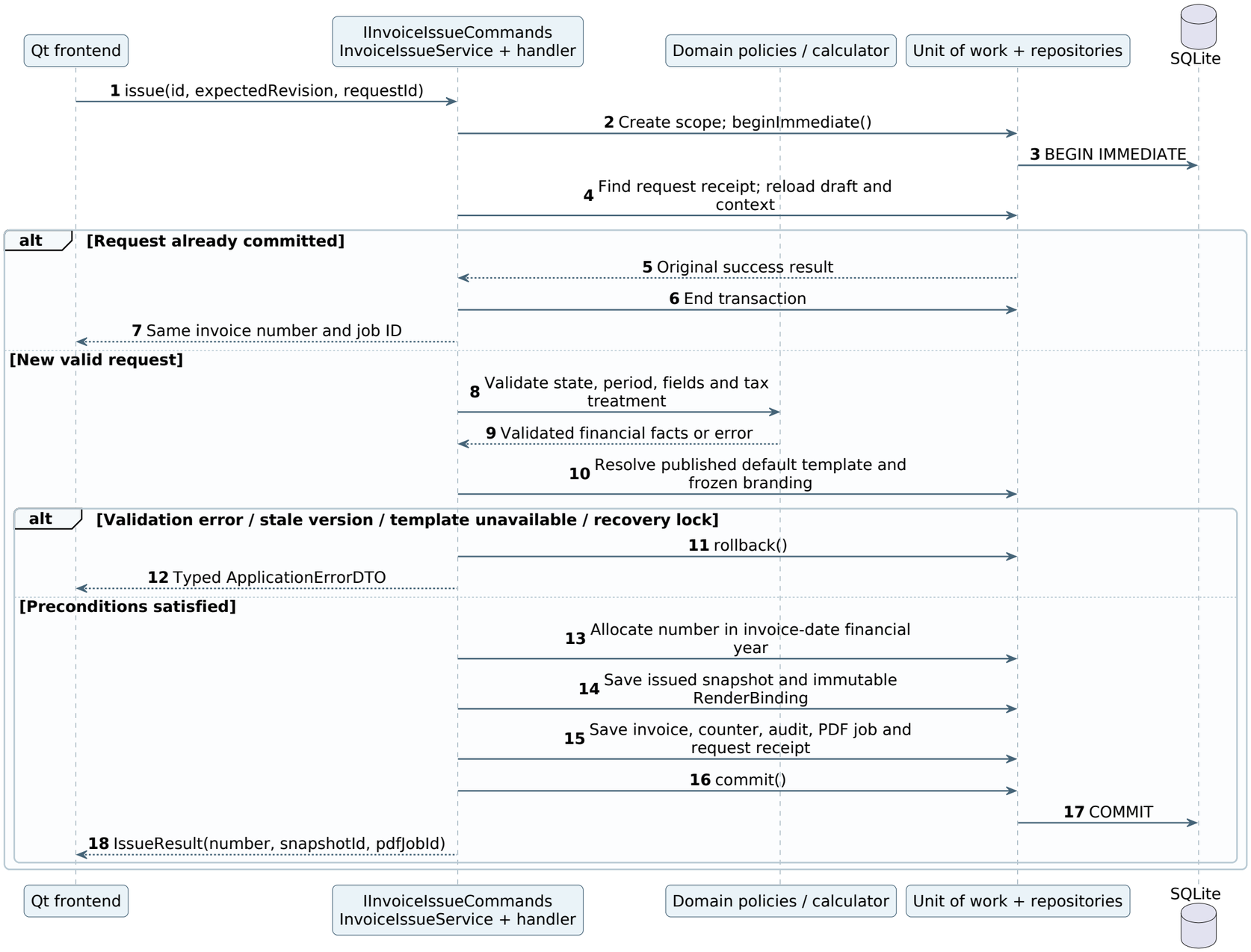
03

The work scope also supplies an idempotency-receipt repository and read-only template/default lookup; these are omitted from the picture to keep it legible. All issued rows, binding, counter, audit, job and request receipt commit or roll back together.

SEQUENCE DIAGRAM

13 Issue once while pinning the exact template version

The transaction fixes both legal facts and document appearance. Rendering is deferred to a durable job after issuance.



01

Number allocation uses the scope (firm, financial year, document kind), with database uniqueness and a transaction. The first number in a new financial year is 1. No clock-triggered reset job or MAX(number)+1 is used.

02

Validation is authoritative in the backend. Issued numbers/dates are not ordinary editable fields. Template selection inside the issuance snapshot prevents an active-template change racing with issuance.

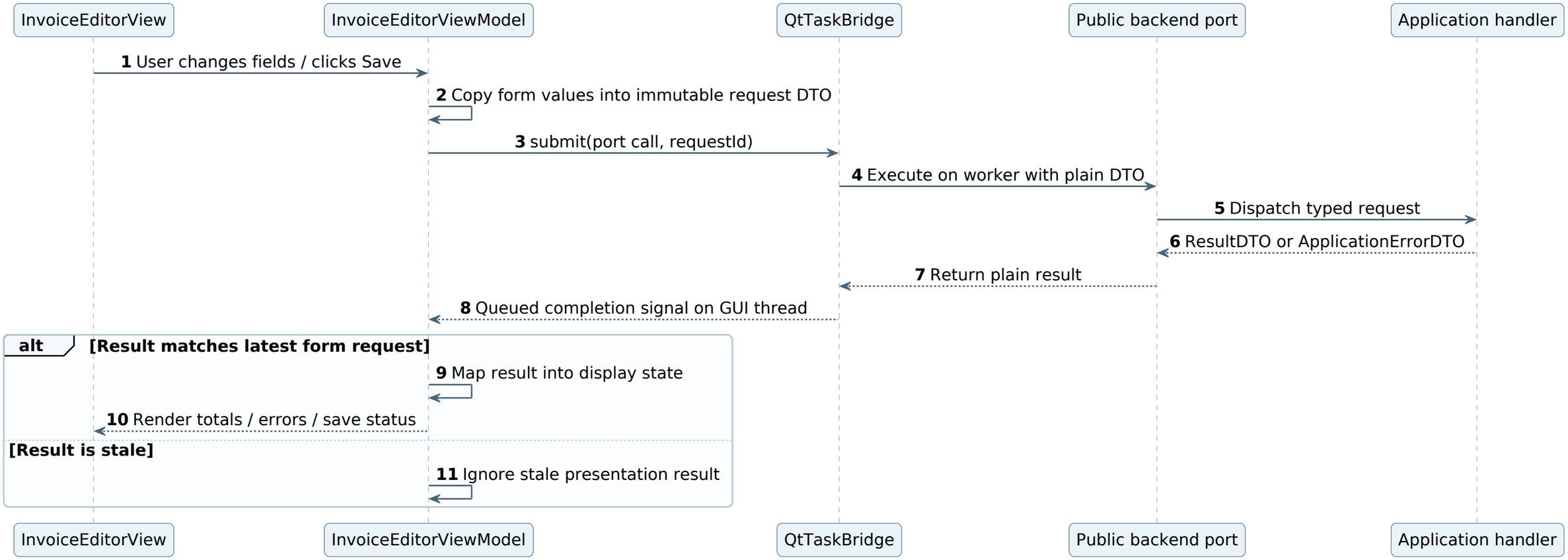
03

The diagram shows the successful commit path. Any database failure rolls back if still active; after an uncertain response, look up the same request ID before retry. PDF failure never reissues the invoice. GST/IRP applicability must be resolved before production release.

SEQUENCE DIAGRAM

14 Frontend/backend interaction without shared UI objects

The backend knows nothing about the window that invoked it; the frontend knows nothing about the repository or renderer implementation.



01

Only QtTaskBridge and view/view-model code use Qt signals, QThreadPool or widget types. The handler is a normal Python object and can execute in a headless test with no QApplication.

02

Frontend cancellation of waiting does not undo a committed invoice. The UI reconciles command results by request ID. Safe job cancellation is a separate backend capability and applies only at defined checkpoints.

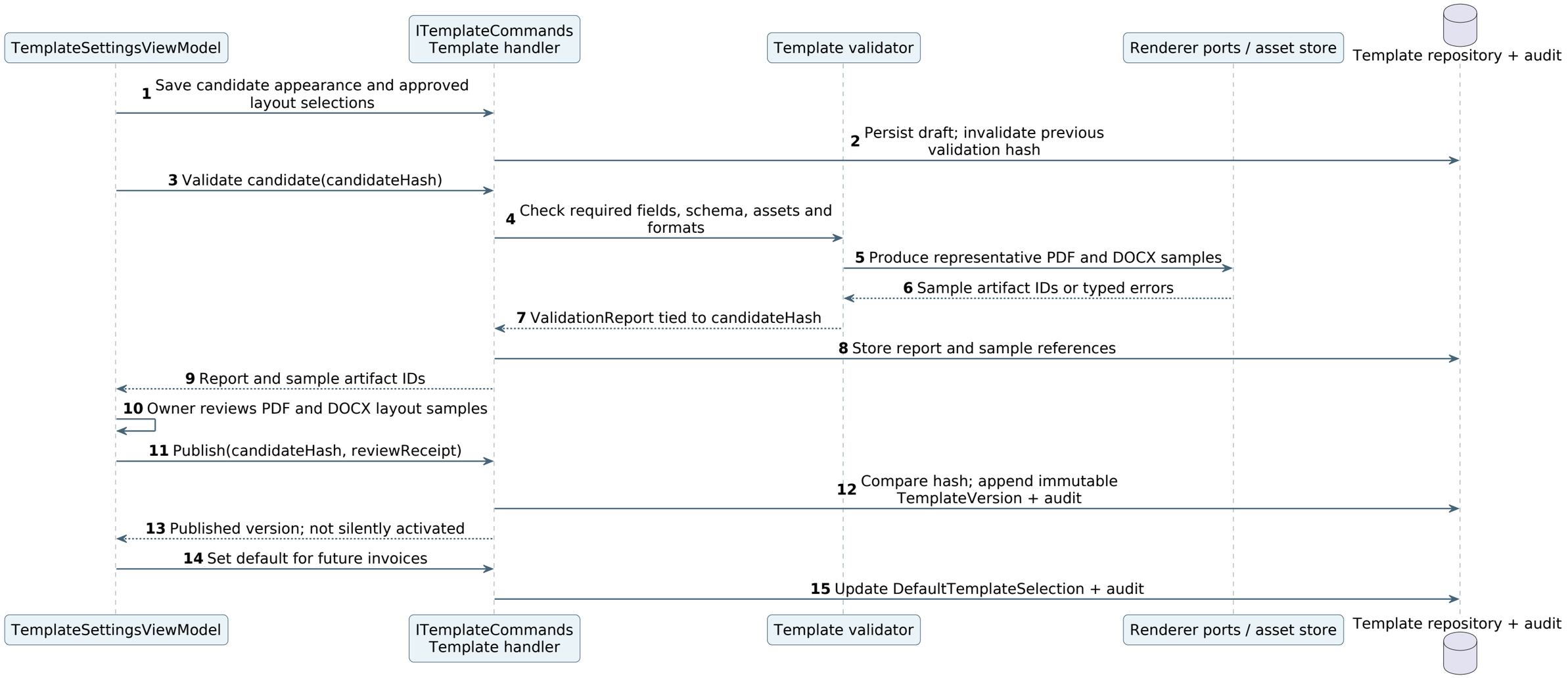
03

Live calculation uses the same public boundary. A small local calculation can execute synchronously when measured safe; all database/file-heavy operations use the bridge so the GUI event loop remains responsive.

SEQUENCE DIAGRAM

15 Editing, validating and publishing a template

Publishing creates a validated immutable bundle; activating it changes which version is selected for future invoice issuance.



01

Template validation checks required invoice fields, compatible model schema, approved fonts and assets, placeholder completeness, long text and multipage item tables. It does not calculate GST or change invoice values.

02

PDF and DOCX samples must be inspected. A DOCX generation library is not a Word layout engine, so visual approval is recorded explicitly. Changing a candidate after validation or review invalidates that evidence.

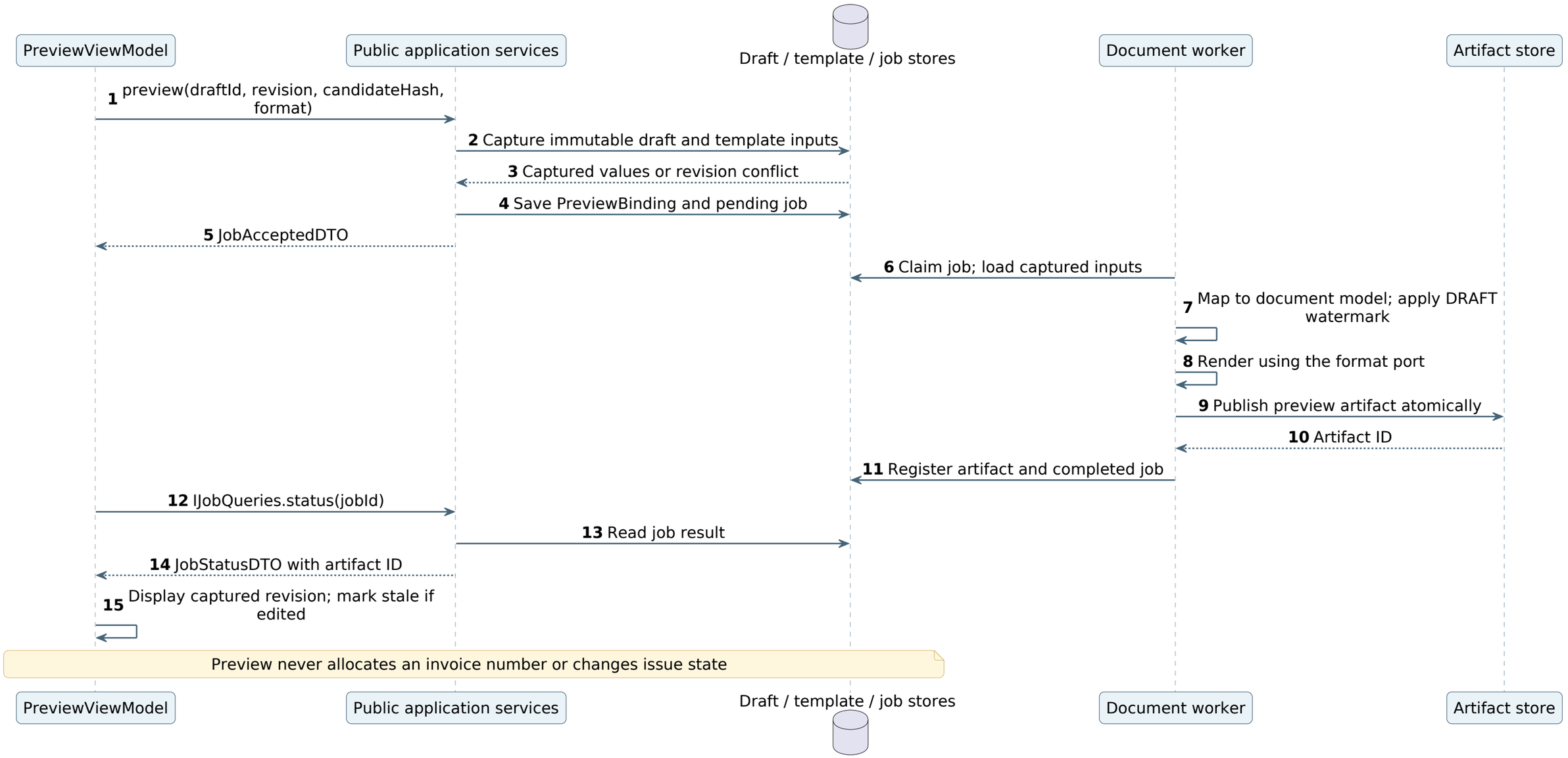
03

Store immutable assets successfully before committing references to a published bundle. A failed metadata transaction may leave removable unreferenced assets, but must not publish a version pointing to missing files. Publishing and activation are idempotent audited commands.

SEQUENCE DIAGRAM

16 Preview through the actual document renderer

Preview follows the same mapper and format adapter as final output, with explicit draft identity and captured template inputs.



01

A preview is not a screenshot of the Qt form. PDF preview opens the generated PDF; DOCX preview opens the generated Word file in the chosen viewer. Embedded preview can be added behind a presentation adapter later.

02

PreviewBinding has captured draft facts, revision, candidate hash and branding but no issued invoice number. Its artifacts have separate retention and keys, carry a DRAFT watermark and never replace canonical issued documents.

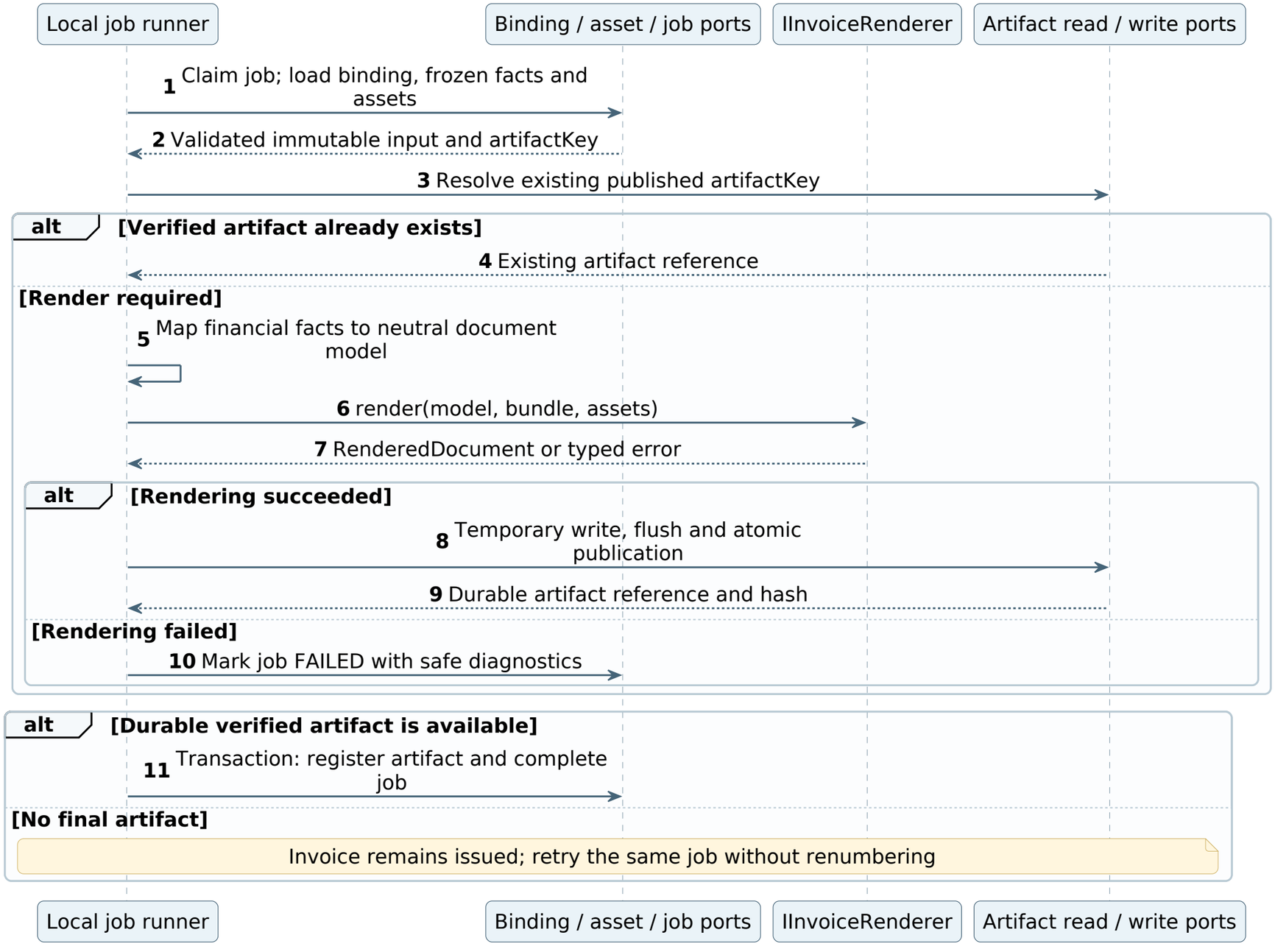
03

Background workers update job records, not view models. The UI obtains plain status DTOs through IJobQueries. If either the draft or template changes, a completed preview is marked stale and can be regenerated.

SEQUENCE DIAGRAM

17 Recoverable rendering through backend-owned ports

The backend publishes durable artifacts and job state. The GUI observes results through public queries; the worker has no GUI reference.



01

A job loads the financial snapshot, published template bundle and branding snapshot through RenderBinding. The mapper prepares a neutral model and the renderer only formats it; neither changes financial facts or invoice state.

02

The unique artifact key includes binding ID, format and renderer version. A crash after file publication but before database registration is repaired by verifying the existing artifact and completing the same job.

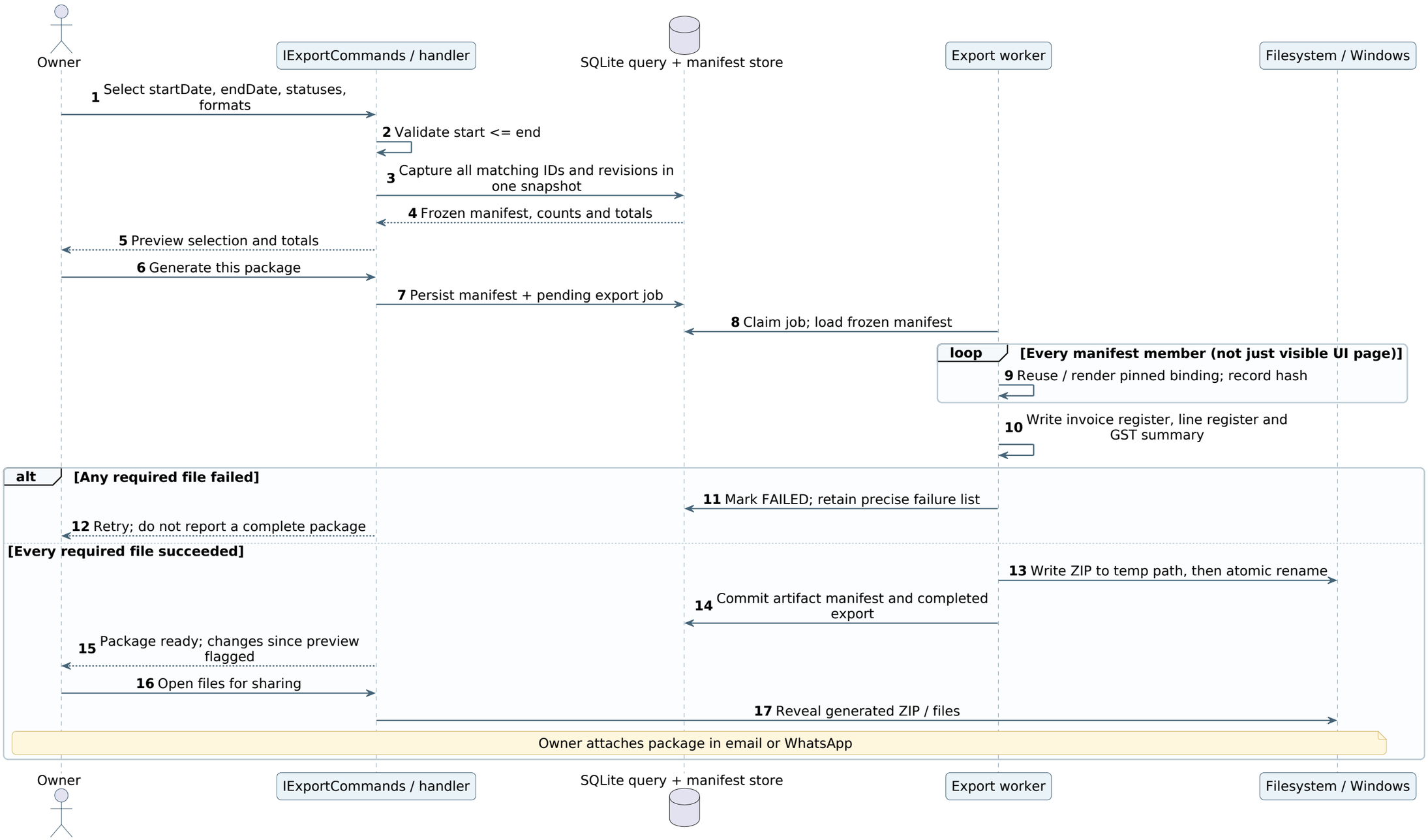
03

File publication and SQLite commit are separate atomic boundaries. Durable jobs bridge them. UI polling or presentation-adapter notifications obtain JobStatusDTO through IJobQueries; there are no worker-to-widget calls.

SEQUENCE DIAGRAM

18 Date-range export with frozen members and bindings

The package matches a frozen selection, including records on every result page and across multiple financial years.



01

Capture every matching record across all result pages using inclusive invoice dates. Export members pin snapshot/binding IDs, selected status, revision and calculated register values. Drafts require an explicit option and a DRAFT label; paid and unpaid issued invoices are both included.

02

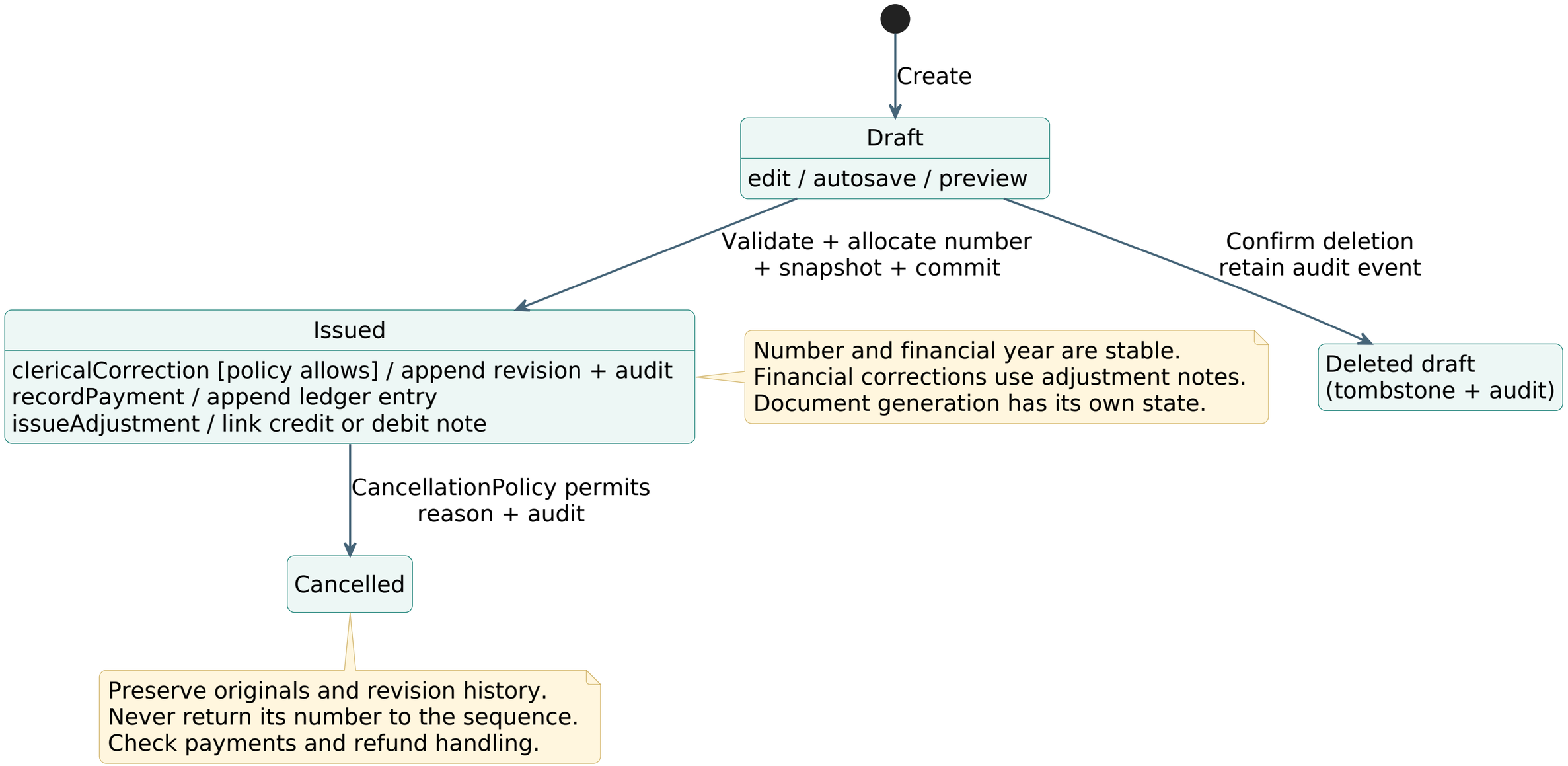
Cancelled originals remain preserved and identifiable in a separate cancellation register. Credit/debit notes are listed by note date and may reference older invoices. Captured manifests make totals and generated documents consistent despite later edits.

03

Separate financial-year folders prevent filename collisions while the printed invoice number stays numeric only. The package contains PDFs, optional DOCX, invoice/line registers and GST summaries. Handoff is not an automated GST filing, upload schema certification or delivery receipt.

19 Invoice lifecycle independent of payment and rendering

Draft editing remains convenient. Issued records preserve traceable history and use explicit correction actions.



01

Deletion is an application operation on drafts, represented by a tombstone rather than silent erasure. Issued invoices cannot be physically deleted through the UI.

02

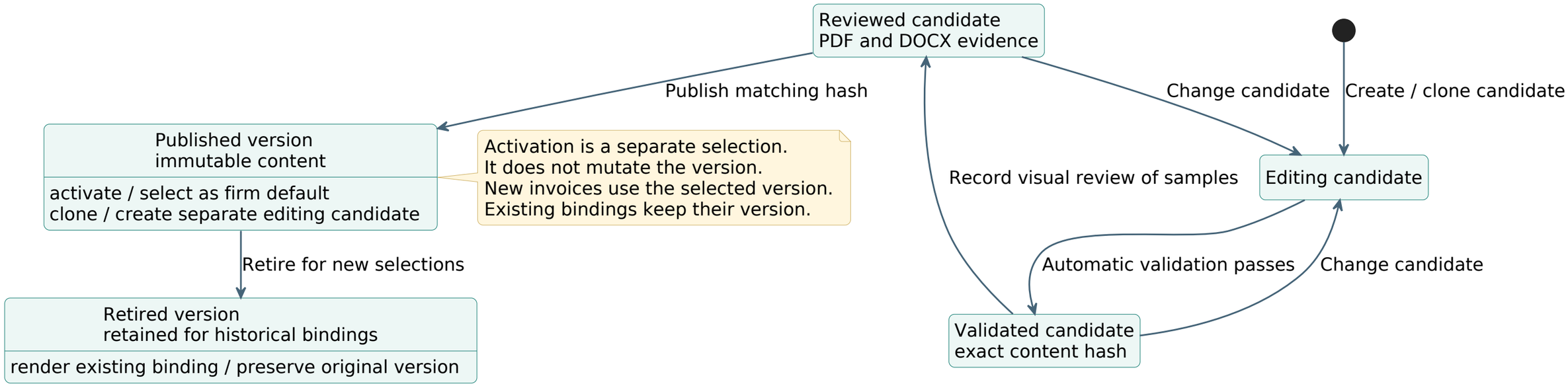
A clerical revision must pass a CorrectionPolicy and keep the original snapshot and artifact. Changes to amount, tax, invoice date across years or identity are not generic edits.

03

Cancellation rules depend on document treatment and filing state; they require a reason and linked settlement handling. External GST/IRP deadlines or permissions must be added if applicable; this lifecycle is not a substitute for those rules.

20 Template lifecycle and immutable publication

Candidate changes are editable; published content is immutable, and default selection is separate from version identity.



01

A candidate cannot be published using a validation report for another hash. Review approval is tied to both PDF and DOCX sample artifacts generated from that candidate.

02

Retiring a version prevents new selection while preserving existing documents. The active default cannot be retired until a valid replacement is selected or issuance is explicitly blocked.

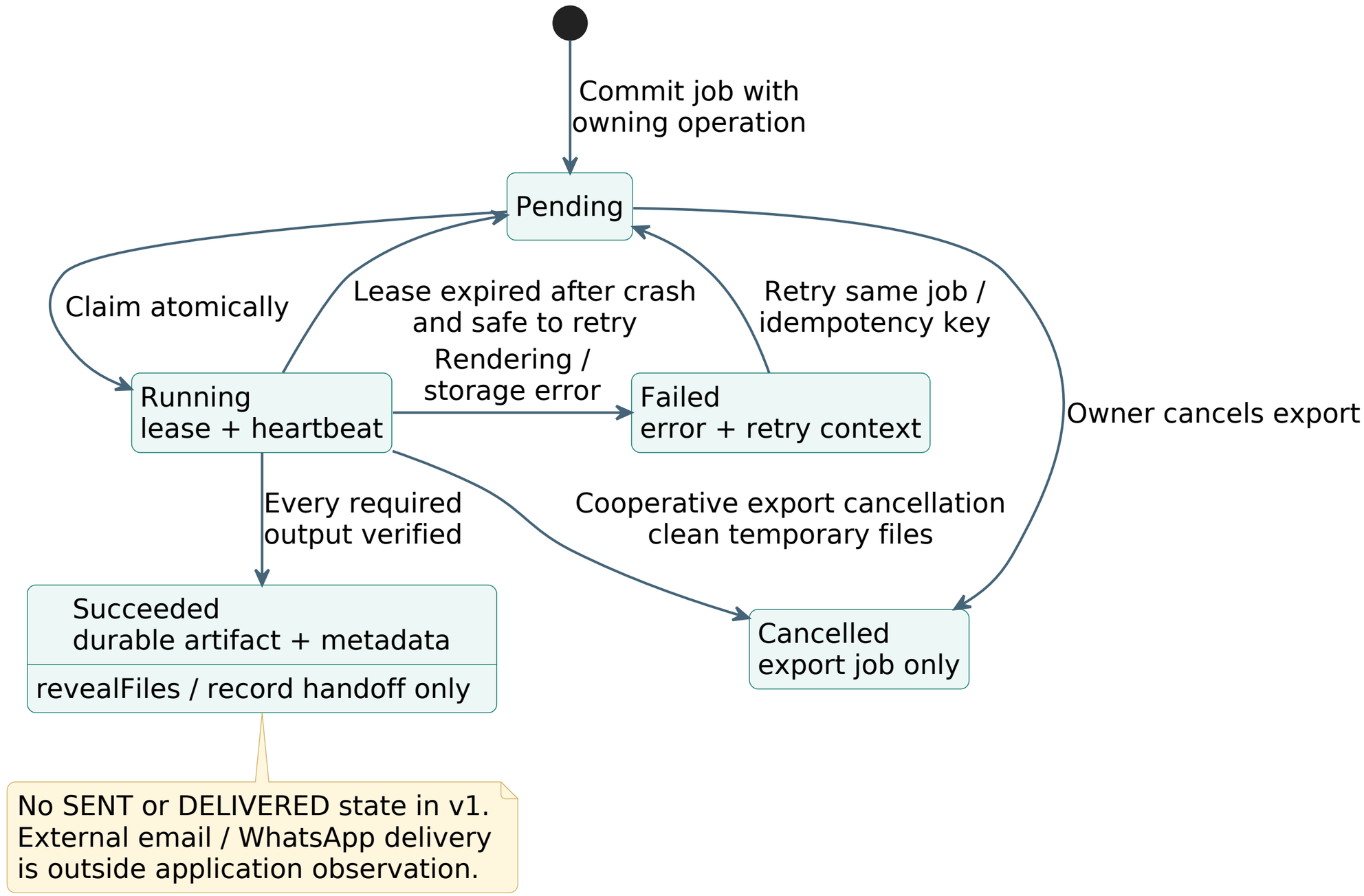
03

Referenced versions, fonts, logos and source layouts are not hard-deleted. A new logo, layout or font selection creates a new version/branding snapshot for new bindings; historical artifacts remain accessible.

STATE DIAGRAM

21 Job lifecycle, retries and handoff semantics

The job lifecycle applies to rendering and export. Backup tasks use the same progress/error concepts but maintain their own backup records.



01

User cancellation of an export never cancels an issued invoice. A pending canonical PDF job is retried rather than silently discarded.

02

Progress and error signals are queued back to the GUI thread. Each worker uses its own database connection; widgets are manipulated only on the main thread.

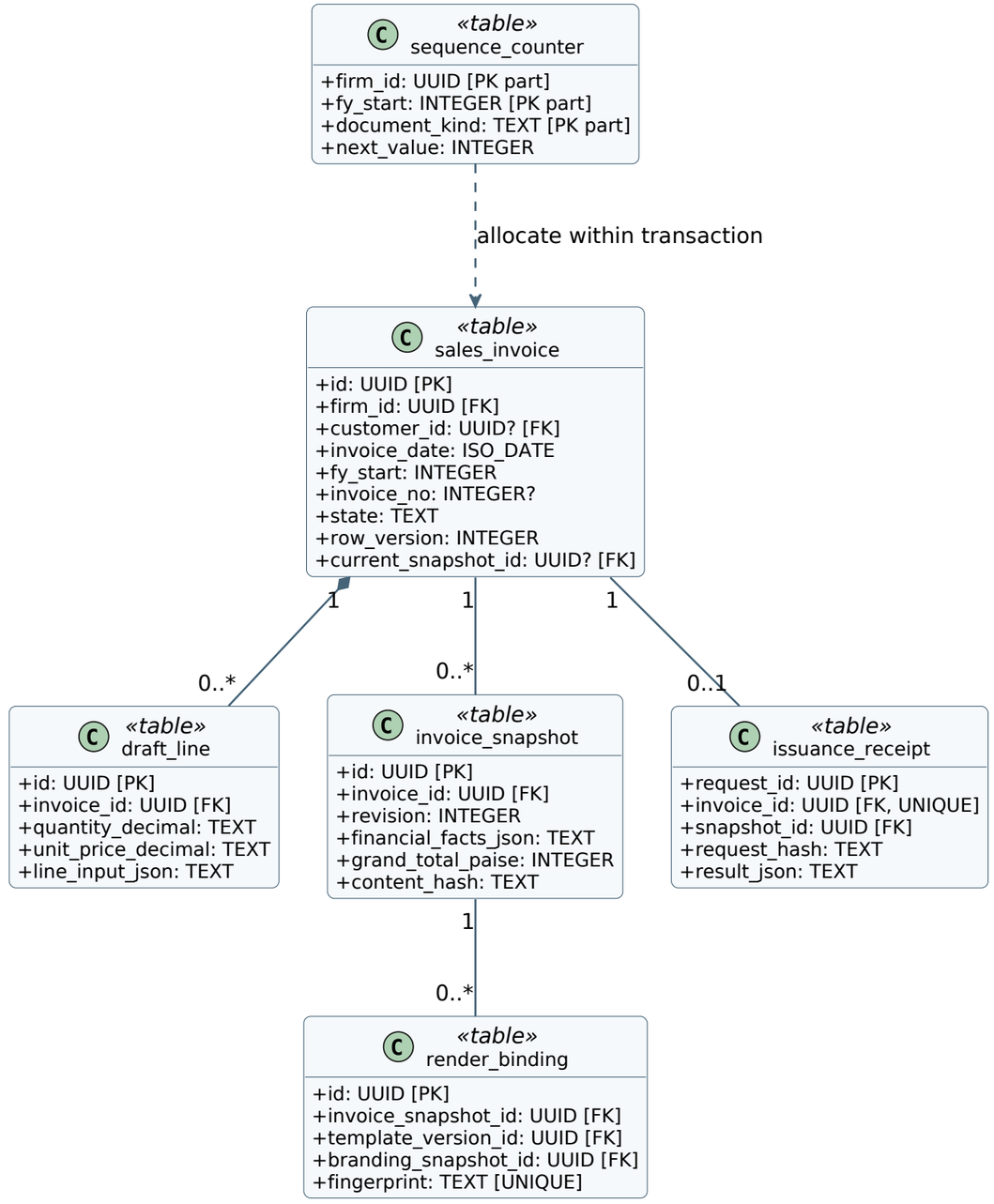
03

Disk-full, permission and template errors are distinct retryable outcomes. Logs avoid full customer details and document content unless explicitly needed for an audited local diagnostic.

CLASS DIAGRAM WITH TABLE STEREOTYPES

22 Invoice persistence and annual numbering constraints

Storage rows are infrastructure models mapped to domain aggregates; the frontend never receives a database row or SQLAlchemy object.



01

Enforce UNIQUE(firm_id, fy_start, invoice_no) WHERE invoice_no IS NOT NULL, positive integer numbers no longer than 16 digits, UNIQUE(invoice_id, revision), and a unique request receipt. Credit/debit note tables use their own document-kind scopes.

02

Enable foreign keys on each connection and index invoice-date/status/customer queries. Use ISO local dates for invoices, UTC timestamps for events, Decimal strings for exact inputs and integer paise for settled money. Reject non-finite numbers and invalid quantities.

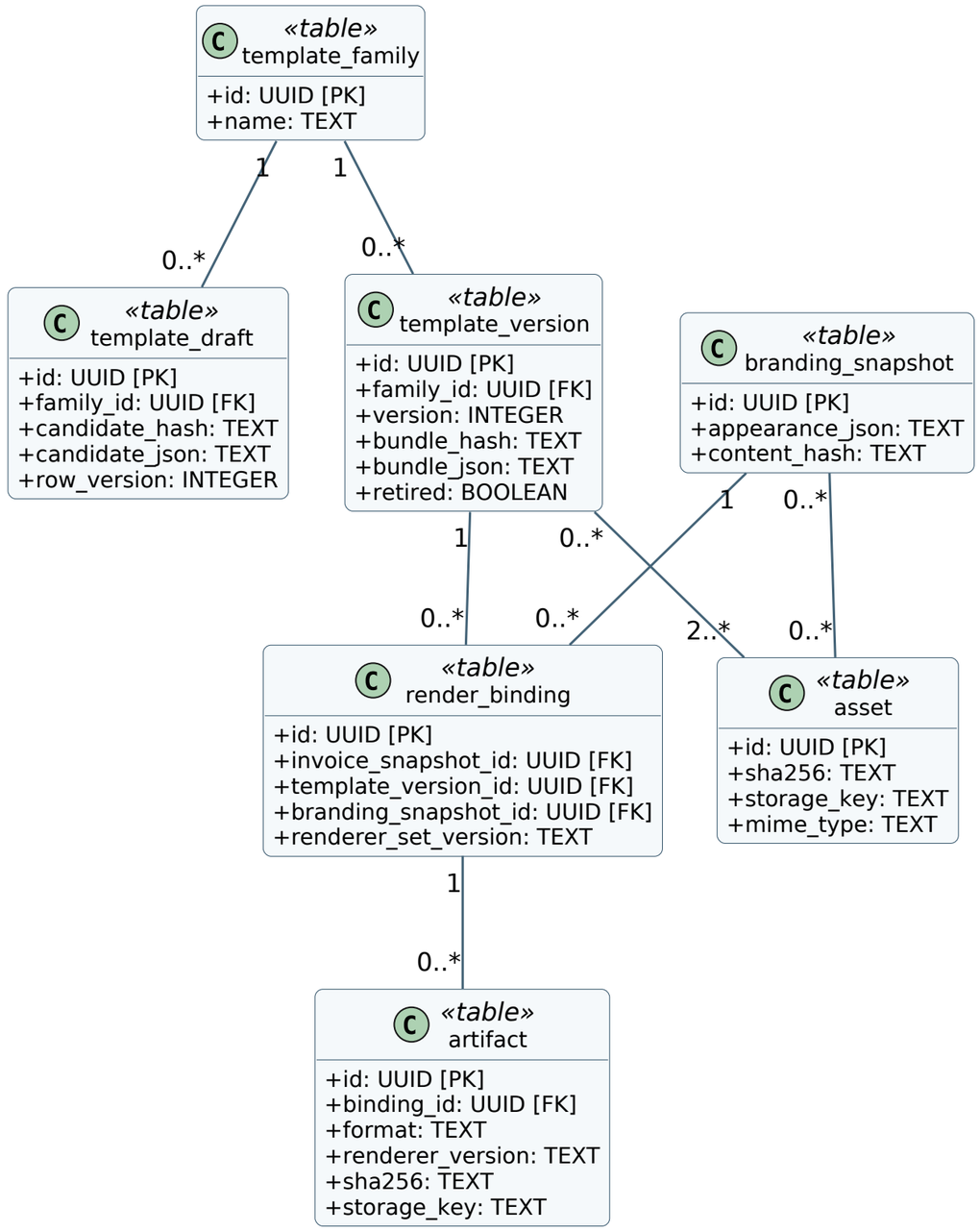
03

SQLAlchemyUnitOfWork performs explicit transactions. Alembic versioned migrations run after a verified pre-upgrade backup. Repository mappers detach domain objects from ORM sessions; optimistic row versions prevent overwriting another edit.

CLASS DIAGRAM WITH TABLE STEREOTYPES

23 Template, branding and document persistence

Templates and branding are durable versioned records, not a mutable file path stored on each invoice.



01

Enforce UNIQUE(family_id, version) and UNIQUE(binding_id, format, renderer_version). Template/branding asset link tables preserve referenced hashes. Active template selection is stored separately per firm and document kind.

02

Additional tables store template_validation_report, template_review_receipt, default_template_selection and preview_binding. A draft preview uses preview_artifact storage with its own lifecycle; nullable polymorphic references must not weaken issued-artifact foreign keys.

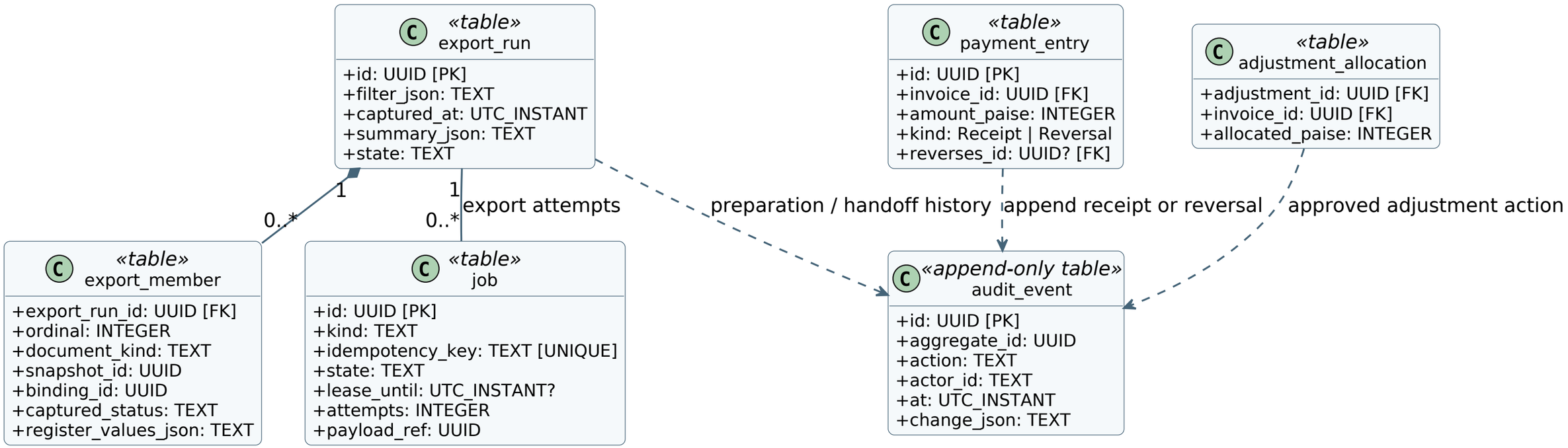
03

Keep template bundles, font assets, logos and canonical documents in the backup manifest. Only unreferenced temporary assets can be cleaned automatically; retirement is not deletion. Filesystem publication is atomic, then metadata references are committed.

CLASS DIAGRAM WITH TABLE STEREOTYPES

24 Export manifests, jobs, payments and audit records

Operational state is separate from invoice identity and template versions. A completed export is a frozen record of what was prepared.



01

Export-member document references are logical references in this view. Implementation uses typed membership tables/foreign keys for sales, adjustments and draft previews, or a checked common document-snapshot index; unchecked polymorphic IDs are not acceptable.

02

Jobs use unique idempotency keys, bounded retries and expiring leases. Audit records capture backend mutations and actor/correlation information. An append-only application interface is not a claim of resistance to local administrator tampering.

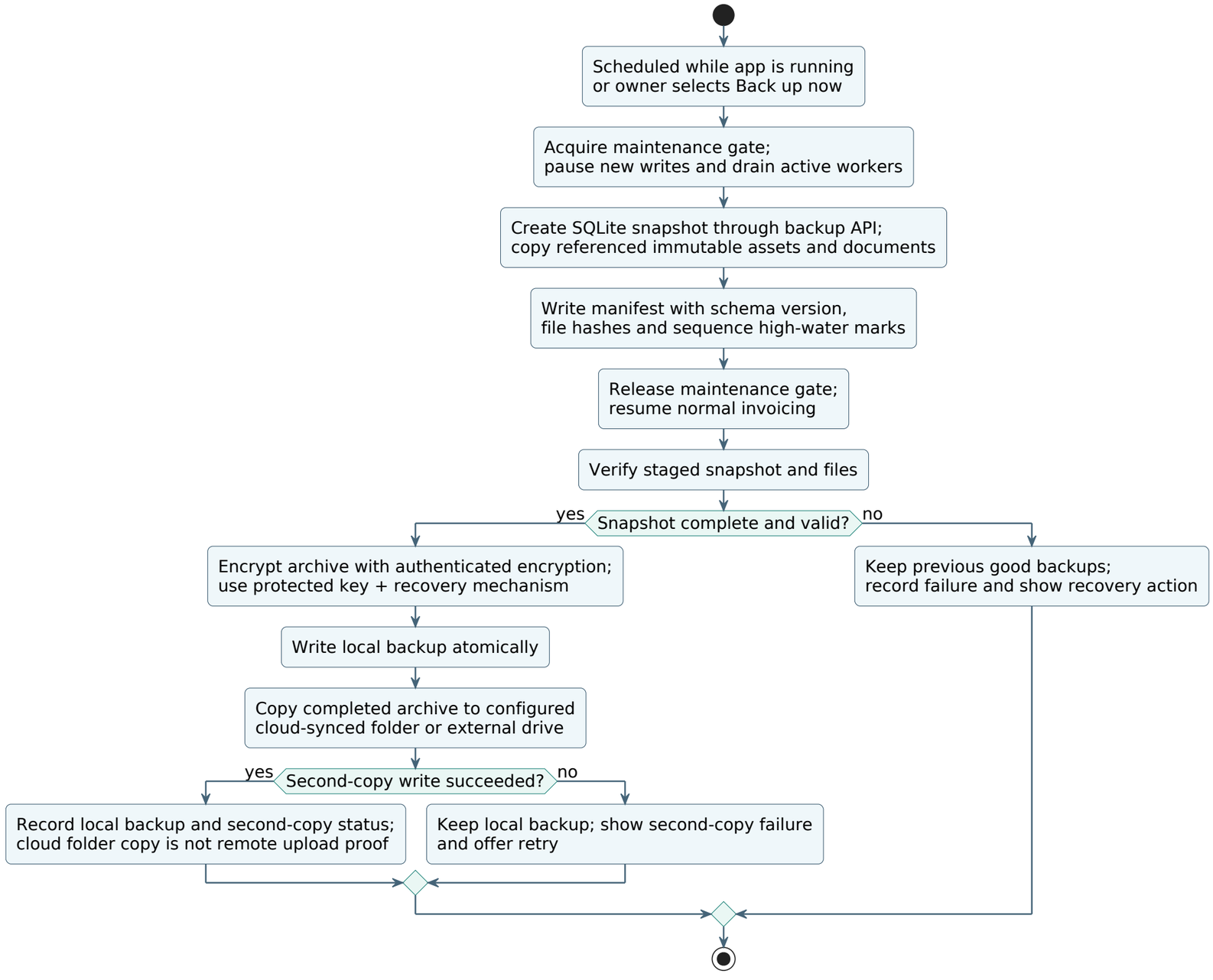
03

Payment status derives from receipts, reversals and allocated credit/debit notes. Track both amount outstanding and any customer credit/refund balance rather than discarding negative net balances. Payments do not change invoice date or tax-register inclusion.

ACTIVITY DIAGRAM

25 Consistent backup including every template dependency

A recoverable backup contains the database plus every referenced document, template and asset, not just an isolated .db file.



01

Pause new writes and drain active workers through a maintenance gate while taking the database snapshot and copying referenced assets. Back up invoice/adjustment records, audit, template versions, branding, fonts, document artifacts, settings and restore metadata together.

02

Use SQLite backup facilities and maintained authenticated encryption with a user-held recovery mechanism. Verify hashes and integrity before marking success. A key protected only for one Windows installation is insufficient for recovery on another computer.

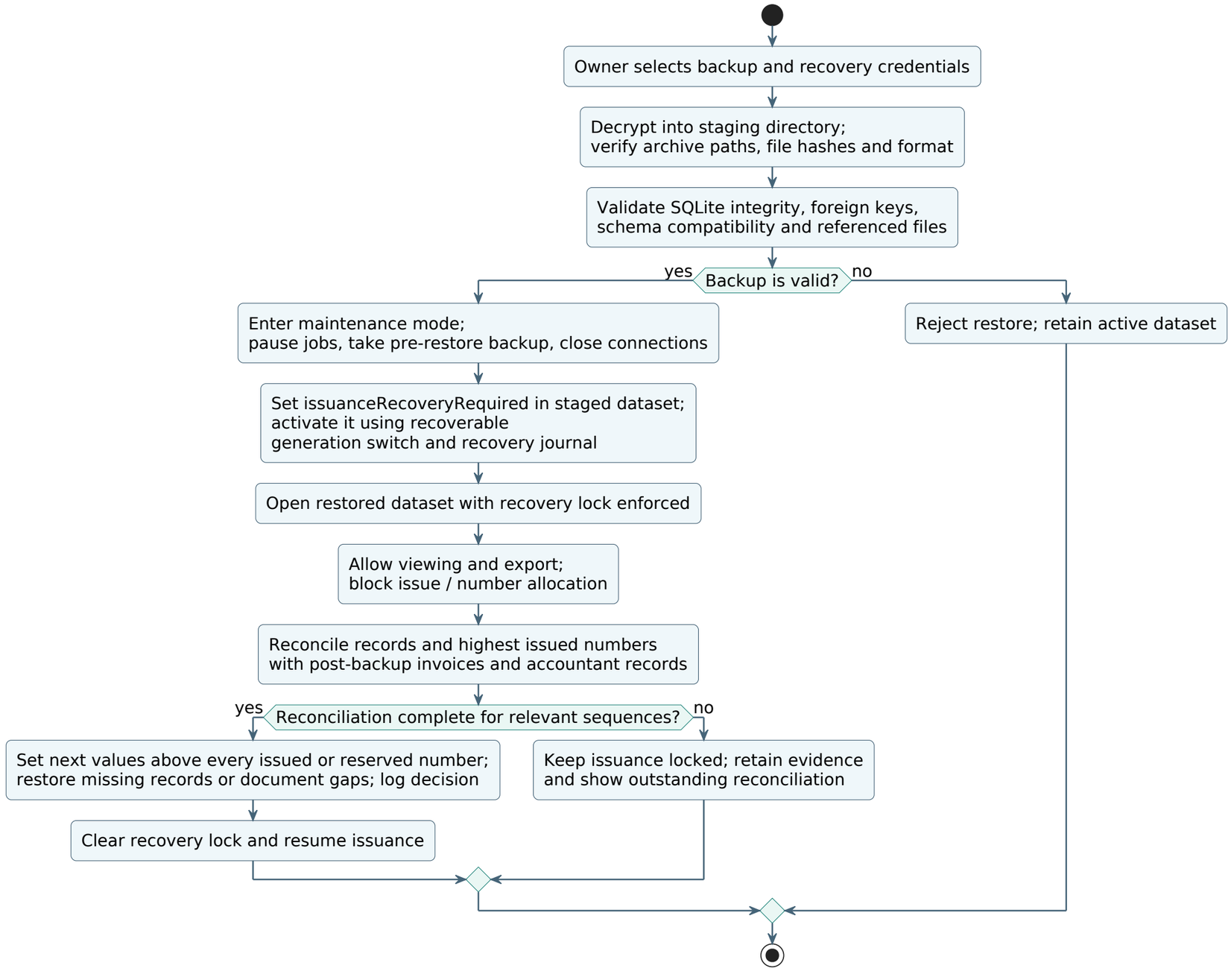
03

Keep multiple dated generations. Copy only completed encrypted archives to the configured cloud-synced folder or external drive. A completed local copy into a sync folder is not proof of cloud upload. Never cloud-sync the active database.

ACTIVITY DIAGRAM

26 Restore with version integrity and numbering reconciliation

Restoring data and resuming invoice issuance are separate operations because a backup cannot reveal invoices issued after it was created.



01

No blind overwrite of a live database, WAL file or artifact directory. A staged dataset is verified first; a recoverable generation-switch manifest makes an interrupted restore diagnosable on next startup.

02

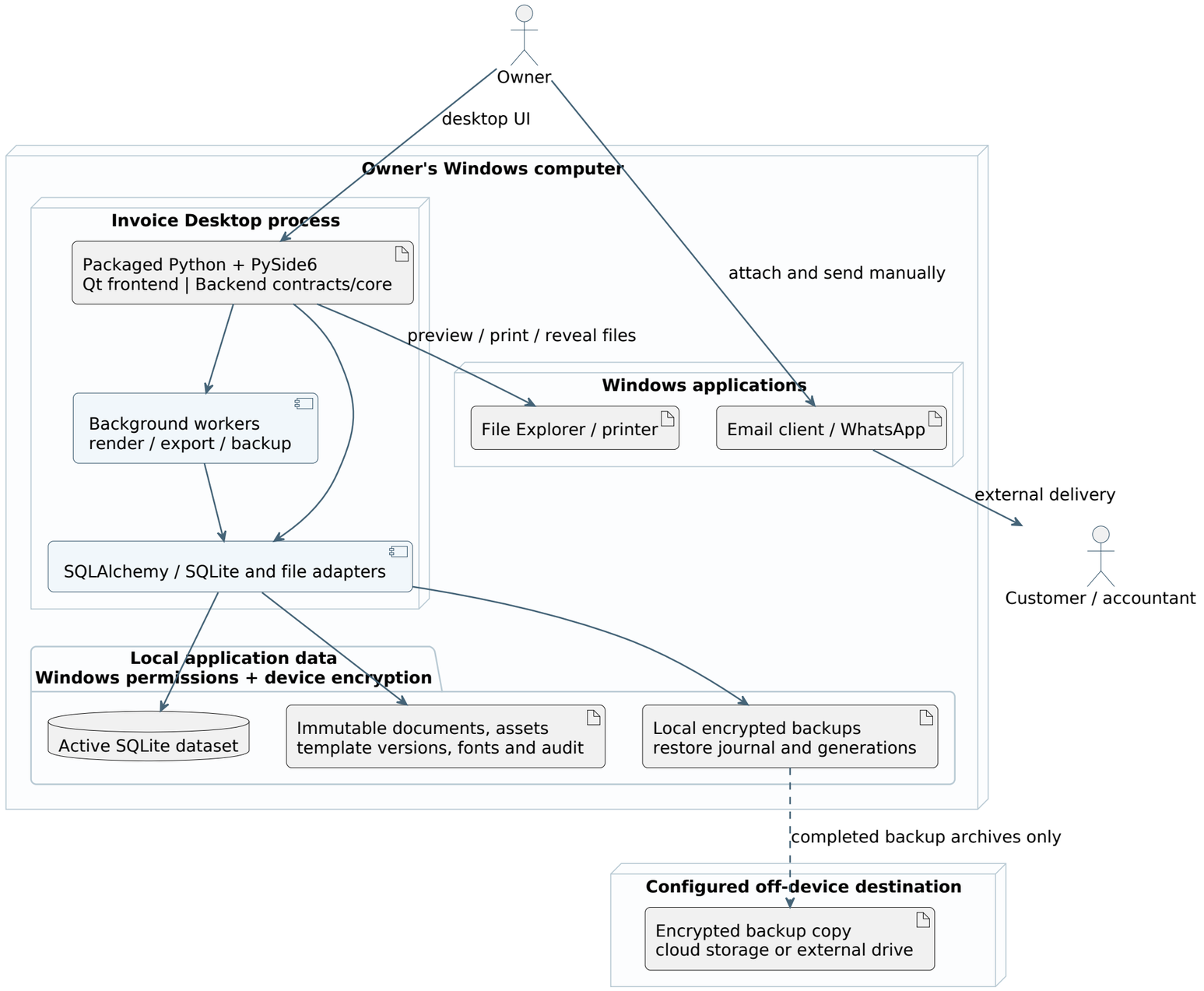
A restore reconciliation checks each financial-year/document-kind counter against evidence of all later issued numbers. If evidence is incomplete, issuance remains blocked; a restored counter alone is not sufficient.

03

Backup key recovery, cross-machine restore, invalid archives, interrupted activation and schema migration are explicit acceptance tests. Logs must not store encryption secrets.

27 Runtime deployment with separate frontend and backend packages

Core invoice operations are local and offline. Internet access is needed only for external messaging or cloud backup sync.



01

Version 1 assumes one principal workstation. The installer and updates keep application binaries separate from user data; upgrades take a backup and run versioned migrations.

02

SQLite is not shared directly between multiple computers. A later multi-computer version introduces an application service and centrally managed database behind repository interfaces.

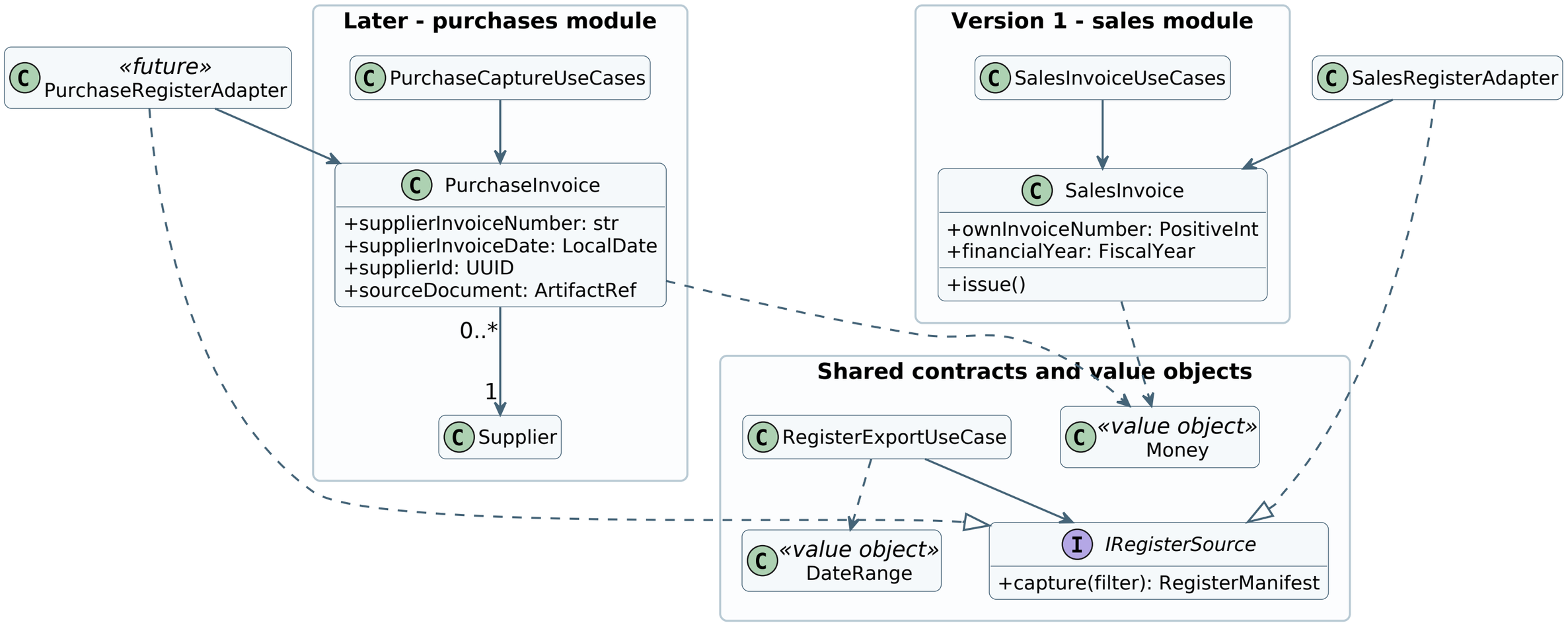
03

Windows permissions and device encryption protect local storage at the operating-system boundary. SQLite is not automatically encrypted by adding an app password. No public cloud link service or automatic message sender is deployed in version 1.

CLASS DIAGRAM

28 Purchase expansion through independent module contracts

Purchase capture becomes a separate module with shared export and value-object contracts, not a subclass forced to behave like an issued sales invoice.



01

A purchase retains the supplier-issued invoice number, which may contain letters and symbols. It does not consume the firm's numeric sales sequence.

02

A minimal InvoiceBase would weaken lifecycle and numbering rules. Separate aggregates preserve valid contracts while shared register adapters permit combined viewing/export later.

03

Supplier management, purchase validation, input-tax-credit review and reconciliation with GST records are explicitly future scope. The current sales app is not a complete accounting or GST-return filing system.

PRINCIPLES MADE CONCRETE

SOLID: design decisions and evidence to require

SOLID is evaluated through responsibilities, dependency direction and behavioural contracts, not by counting interfaces.

Principle	Where it appears	Evidence to require in the code
S - Single responsibility	Views display; view models map input/state; handlers orchestrate; domain services calculate; repositories persist; mappers prepare document data; renderers format; template services publish versions.	Change an invoice layout without changing InvoiceCalculator or numbering. Change a screen without changing repositories. Audit mutations independently of diagnostic logging.
O - Open / closed	Add a renderer implementation and register it in bootstrap. Add a tax-treatment policy or template bundle through a defined extension point. New purchase capture is a separate module.	Existing issue/draft handlers do not acquire format-specific branches for each renderer. Wiring changes are expected at the composition root.
L - Liskov substitution	Every IInvoiceRenderer preserves the same invoice facts, required fields, failure contract and absence of side effects. Every repository honours concurrency and transaction semantics.	Run the same adapter contract suite against each renderer/repository. A fake renderer may simplify layout but cannot invent totals or omit required facts. PurchaseInvoice is not a SalesInvoice subclass.
I - Interface segregation	Separate draft commands, issue/correction commands, queries, template commands/queries, jobs, artifact reads/writes and persistence responsibilities.	An invoice list cannot access invoice mutation through its query port. A template screen does not receive payment operations. Narrow repositories are bundled for a transaction without merging their responsibilities.
D - Dependency inversion	Handlers depend on ports; infrastructure implements them. Presentation depends on application.contracts. bootstrap alone selects concrete implementations.	Domain and application tests run without QApplication or renderer libraries. No view imports SQLAlchemy, no domain object imports Qt, and no handler instantiates a concrete database or renderer adapter.

COMPOSITION WITHOUT A GOD OBJECT

Application services implement cohesive public ports and delegate to individual handlers. InvoiceIssueService supplies issue, correction and cancellation through separate handlers; IssueInvoiceHandler itself is not claimed to implement unrelated methods. IssuanceWorkScope groups transaction-bound dependencies only. There is no all-purpose InvoiceManager that calculates, saves, renders, opens windows and sends messages.

Public API catalogue and error semantics

All operations return typed values or typed errors. The UI does not rely on exceptions from third-party libraries.

Capability	Operations	Result / invariant
IInvoiceDraftCommands	save, duplicate, delete draft	New/changed DraftDTO and revision; duplicate has a new UUID and no invoice number.
IInvoiceIssueCommands	issue, approved correction, cancel	IssueResult or CommandResult; original identity preserved where applicable; request IDs are idempotent.
IInvoiceQueries	get, paged list, calculateDraft	Immutable detail/page/calculation DTOs. Calculation is authoritative in the backend.
IMasterDataCommands / Queries	firm profile, customers, catalogue	Master DTOs; revisions or validation errors. Updating masters does not change issued snapshots.
ITemplateCommands / Queries	save, validate, publish, activate, retire, list	Candidate/version/report DTOs. All evidence is tied to content hashes.
IDocumentCommands	preview draft, generate issued format	JobAcceptedDTO. Numbering is never part of a render request.
IExportCommands	prepare selection, generate captured manifest	ExportPreviewDTO then JobAcceptedDTO. No dependency on UI pagination.
IJobQueries / Commands	status, retry, safe cancellation	JobStatusDTO; terminal states have explicit success/failure. Cancellation never silently rolls back a committed invoice.
IPaymentCommands / Queries	record receipt, reverse entry, get balance	Payment/BalanceDTO; receipt/reversal history retained; outstanding and credit balances are distinguished.
IBackupCommands / Queries	start backup, validate restore, restore, status	Job/status DTOs; restore is a controlled maintenance operation with recovery-number reconciliation.
IShareCommands	reveal approved artifact IDs	HandoffResultDTO, not a delivery receipt. Backend resolves and validates file access.

ERROR CODES AND PRESENTATION

VALIDATION carries field errors; CONFLICT carries the current revision; NOT_FOUND identifies an unavailable resource; TEMPLATE_UNAVAILABLE or UNSUPPORTED_FORMAT prevents a misleading export; UNSUPPORTED_TAX_TREATMENT blocks guessed calculations; STORAGE_FAILURE and JOB_FAILED expose safe retry guidance; RECOVERY_REQUIRED blocks issuance after restore. Each error carries a correlation ID. Logs hold technical diagnostics without exposing secrets to the UI.

Allowed imports and architecture checks

These rules turn the diagram into a boundary that can be checked automatically when code exists.

Package	Allowed dependencies	Forbidden responsibilities / dependencies
presentation.qt	application.contracts; its own UI modules; Qt and standard library	application.handlers, domain, infrastructure, SQLAlchemy, ReportLab, docxtpl
application.contracts	Standard-library dataclasses, enums, typing and value primitives	Qt, domain aggregates, handlers, infrastructure, persistence and renderer libraries
domain	Its own modules; standard library; pure policy interfaces	presentation, application, infrastructure, database or document packages
application.handlers	contracts, outbound ports, domain and pure mapping helpers	presentation, Qt, concrete SQLAlchemy/file/renderer adapters
application.ports	Neutral document/DTO contracts; domain types when repositories need them	presentation, concrete handlers, infrastructure and framework-specific types
infrastructure	Application ports/contracts; domain mappers; relevant third-party libraries	presentation and concrete use-case handlers
bootstrap	Public contracts, view-model constructors, handler constructors and concrete adapters	Business logic; persistent mutable globals used as service locators

ENFORCEMENT IN THE IMPLEMENTATION

Use import-linter forbidden/layer contracts, including external packages where supported. Add an architecture test that inspects public method annotations for Qt, ORM or renderer types. Start the backend tests in an environment where PySide6 is absent. Treat failing boundary tests as build failures.

WHY A DESKTOP APP CAN STILL BE DECOUPLED

The frontend and backend can run in the same process while remaining independent packages. The separation comes from contracts and ownership. Network transport is not necessary for this first release; if later required, an HTTP/IPC adapter can be introduced without making the domain depend on that transport.

APPEARANCE WITHOUT BUSINESS-LOGIC COUPLING

Document template design and customisation

The application interface and the invoice document are different design surfaces with different configuration.

Design area	Owned by	What it controls
Application screens	Qt Designer / PySide6 widgets	Forms, tables, buttons, navigation and dialogs. UI style settings are local presentation concerns.
Invoice header	FirmBlock + frozen legal details + branding assets	Logo, firm identity and contact arrangement. The template consumes stored details; it cannot substitute a different firm/GSTIN.
Buyer and invoice metadata	PartyBlock + InvoiceMetaBlock	Customer, invoice number/date and relevant supply information. Numbers remain numeric only.
Item table	ItemRow tuple with approved column schema	Description, HSN/SAC, quantity, unit, rate, discounts and tax details. Repeating rows, wrapping and repeated headings must work across pages.
Totals and amount in words	TotalsBlock from frozen invoice facts	Taxable value, tax components, rounding, grand total and words. All formats receive the same facts; no template formulas.
Footer	TermsAndPaymentBlock + approved optional sections	Bank/payment details, terms, notes and signatory area. Business content is snapshotted; decorative layout is versioned.
PDF layout	ReportLab adapter + approved layout descriptor	Stable A4 print layout, fonts, spacing, borders, totals positioning and pagination.
Word layout	docxtpl adapter + approved .docx template	Placeholders, repeating item rows and images. Strict missing fields and XML escaping are mandatory.
Settings and history	AppearanceConfig + TemplateVersion + RenderBinding	Approved font, accent colour, margins, optional sections, logo/signature references. Publication and activation are separate audited operations.

MATCHING DATA, WITH HONEST FORMAT LIMITS

PDF and Word must agree on all invoice facts and approved branding. Word pagination depends on its renderer and available fonts, so identical pixels/page breaks are not assumed. Template publication includes both sample files and a visual review. The archived canonical PDF remains the issued visual record. A later single DOCX-to-PDF engine would be a separate renderer choice with its own runtime dependency.

Transactions, side effects and recovery guarantees

These guarantees are enforced in handlers, repositories and constraints rather than only in the form.

Boundary	Guarantee
Issue transaction	Validate draft and context, resolve a usable template/default, allocate annual number, save issued facts and binding, append audit, enqueue PDF job and save request receipt in one database transaction.
Numbering	Scope is firm + financial year + document kind. First allocation in a new year is 1. Use write serialization and uniqueness; never MAX+1. A cancelled number remains reserved. Backdated or closed-period changes use explicit policy.
Monetary precision	Construct Decimal from validated decimal strings; reject non-finite values. Apply a tested rounding policy and store final values as integer paise. Words derive from the final rounded total.
No slow I/O in issue transaction	No PDF generation, DOCX generation, network call or long file copy inside issuance. Published assets already exist; metadata is pinned. Short database transactions minimise contention.
Rendering boundary	A durable job renders a pinned binding and atomically publishes a file. Artifact registration is a separate database transaction. Retry detects an existing hash-verified artifact instead of generating another invoice.
Template publication	Validation and review receipts match the candidate hash. Publish immutable assets before metadata references. A default-template switch affects future issuance only.
Export consistency	Capture all member IDs, revisions, bindings, statuses and register values together. The package uses that manifest. If any required member fails, report failure rather than claiming a complete package.
Restore boundary	Verify a staged dataset, take a pre-restore backup, close active connections and activate through a recoverable generation journal. The staged dataset carries an issuance lock before activation.
Recovery numbering	Reconcile post-backup issued/reserved numbers and missing records against external evidence before clearing the restore lock. A restored next_value alone cannot prove that a number is unused.

TRACEABILITY

Functional requirements: responsibility and coverage

F01-F30 refer to the agreed/proposed functional list in this conversation; template requirements are added explicitly.

IDs	Requirement	Owner in the design	Views
F01-F03	Sales invoices, goods/services and multiple items	Invoice aggregate, editor commands and DTOs	01, 03, 04, 06, 08
F04-F05	Automatic calculations and INR words	InvoiceCalculator, approved tax/rounding policy, InrWordsFormatter	07
F06-F08, F18	Numeric-only annual numbering and historical access	Issue handler, sequence repository, fiscal-year uniqueness and snapshots	06, 12, 13, 22
F09-F12, F22, F24	Store/view/edit/delete, drafts and duplication	Draft service, repositories, optimistic revisions, lifecycle and audit	03, 04, 19, 22
F13-F14, F25	Inclusive dates, search and range totals	Query ports, typed filters and backend read models	04, 18, 24
F15-F17, F23	PDF/DOCX, bulk export, print/preview and sharing	Document/Export/Share ports, adapters, artifact IDs and Windows handoff	05, 10, 11, 16-18, 27
F19-F21	Reusable firm, customer and catalogue records	Master-data handlers and value copying into snapshots	01, 06, 08
F26	Accountant register and GST summary	Frozen export manifest, register writers and separate adjustment/cancellation registers	18, 24
F27	Payment tracking	Receipt/reversal ledger, adjustment allocations and derived balance	08, 19, 24
F28-F30	Change/export history, access, backup/restore	Audit log, backend actor/capability context, complete encrypted backups	08, 24-27
T01-T04	Configure templates, real preview, validate and publish	Template commands, domain records, preview bindings and two-format review	05, 09-11, 15, 16, 20, 23
T05-T07	Activate/retire versions, preserve history, separate UI theme	Default selection, immutable bindings and package ownership	02, 09, 10, 20, 23
Future	Purchase capture and supplier-issued numbering	Separate purchase module and shared neutral register contracts	28

Non-functional requirements and measurable checks

Proposed budgets must be verified on the target Windows computer; no performance measurements are claimed here.

IDs	Attribute	Acceptance evidence
N01 / N14	Usability and accessibility	Keyboard traversal, readable labels, field-specific errors, predictable save/preview actions, responsive tables and explicit destructive-action confirmation.
N02	Financial accuracy	Golden examples for GST treatment, discounts, rounding and words; exact agreement of amounts across PDF, DOCX, Excel and database records.
N03 / N04	Data integrity and historical consistency	Transactional issuance, uniqueness, optimistic concurrency, immutable snapshots/bindings, safe duplicate handling and audit records.
N05	Security and privacy	OS user permissions/device encryption, backend capability checks, validated file paths, controlled template assets and encrypted backup recovery. No passwords or full documents in routine diagnostics.
N06	Reliability and recovery	Crash-safe jobs, bounded retries, verified artifacts, recoverable restore journal, and successful restore rehearsals including a lost device.
N07	Performance	Initial target: common filtered queries within 2 s at 10,000 invoices; an ordinary PDF within 5 s. Benchmark representative line counts and cold/warm runs. Large jobs expose progress/cancellation.
N08	Export quality	Long names/descriptions, one and many item rows, multipage headers, totals and signatures remain legible. Inspect Word in the intended viewer; do not infer DOCX layout from its XML.
N09	Reporting consistency	Manifest count equals included required documents. Invoice/line/tax summary values reconcile; cancelled/draft/adjustment treatment is explicit.
N10	GST suitability	Required fields and approved tax treatments are validated in the backend. E-invoice applicability, special supplies and correction policies must be resolved before the production issuance flow is enabled.
N11	Maintainability	Enforced import rules, narrow ports, typed contracts, versioned migrations and separate interface/adapter tests.
N12	Extensibility	A new renderer is registered behind a contract. Purchase capture gets its own lifecycle and numbering. Multi-computer use introduces a service boundary rather than a shared SQLite file.
N13	Compatibility and portability	Packaged Windows build, standard PDF/DOCX/XLSX/CSV outputs, portable full-backup format and migration/export paths that preserve identifiers and history.

TEST PLAN

Acceptance scenarios that expose coupling and data loss

These tests are required for implementation; this document validates diagram syntax and layout only.

Scenario	Exercise	Expected result
Frontend substitution	Run draft calculation, save, issue and export through public ports with no QApplication or installed PySide6.	No GUI or presentation imports occur in the backend execution path.
DTO boundary	Inspect type annotations and runtime returns from every public method.	No QObject, QWidget, SQLAlchemy Session/model, ReportLab Flowable or docxtpl object crosses the boundary.
Renderer substitution	Run the same input model against PDF and DOCX adapters and a fake renderer.	All invoice facts match; side effects are absent; errors use the declared contract.
Template version change	Issue with template A, activate template B, then reopen/export the old invoice.	The original binding and artifact still use A; new issuance uses B.
Branding change	Change logo/font and generate a new invoice; re-export an old one.	Distinct binding/artifact keys; no overwrite or retroactive appearance change.
Preview correctness	Edit a draft or template while its preview is rendering.	Preview is tied to the captured revision/hash, labelled DRAFT, and marked stale when appropriate; no number allocated.
Template publication	Validate candidate hash A, edit to hash B, then attempt publication with A evidence.	Publication rejects stale validation/review; required-field omissions and missing fonts fail clearly.
Double issue / lost response	Double-click Finalize, race two issue commands, and lose the success response.	One issued invoice and one sequence allocation; retry retrieves the existing receipt.
Annual sequence	Issue on 31 March and then 1 April; export across both years.	New year starts at 1; durable IDs stay distinct; files cannot collide; display numbers contain no year prefix.
Document crash recovery	Fail after file publication but before artifact registration; fail again with disk full.	Safe recovery registers a verified existing file; no renumbering; failed files are not exposed as complete.
Export completeness	Use more rows than one UI page and change a record after preview.	All captured members are exported; totals match the manifest; subsequent changes are flagged.
Restore and recovery key	Restore an older encrypted backup on another computer.	All assets/templates resolve; keys are recoverable; issuance stays locked until later numbers and records are reconciled.

Technology mapping and implementation order

The architecture fixes responsibility and contracts while allowing adapters to be replaced.

Layer	Technology	Responsibility
Presentation	PySide6 Qt Widgets + Qt Designer; QtTaskBridge	Screens and background-call integration. Application UI styling stays within presentation.
Public API / domain	Python frozen dataclasses, typing.Protocol, enums and Decimal	Plain DTOs, interfaces, aggregate rules and exact calculation. No GUI/database dependency in domain.
Persistence	SQLite + SQLAlchemy adapters + Alembic	Repository implementations, explicit transactions, read models, constraints and versioned database upgrades.
Document adapters	ReportLab; docxtpl/python-docx	PDF and Word from neutral document models and published template bundles.
Registers and packages	openpyxl, csv, zipfile	Excel/CSV accountant registers and complete ZIP packages; generated from captured export members.
Backup / diagnostics	SQLite backup API, cryptography, Python logging	Consistent encrypted backups, recovery tooling, safe diagnostic logs distinct from business audit history.
Checks and delivery	pytest, pytest-qt, import-linter; pyside6-deploy; Windows installer	Domain, contract, integration and GUI tests; dependency checks; packaged executable and installer.

IMPLEMENT IN VERTICAL SLICES

1. Contracts, domain calculations and headless tests. 2. SQLite repositories and transactional numbering. 3. One complete draft-to-issued-PDF workflow. 4. Template settings, versioned DOCX/PDF bundles and preview. 5. Search, date-range exports and accountant registers. 6. Payment/correction history, backups, recovery and packaging. Run boundary checks throughout.

OPEN BUSINESS DECISIONS BEFORE PRODUCTION

Confirm firm details and preferred invoice layout, approved tax/rounding cases, e-invoicing applicability, opening invoice numbers and historical imports, correction/closed-period rules, owner permissions, backup destination and recovery-key ownership. These do not prevent implementing the decoupled foundation, but they affect production issuance policies.

REFERENCE

Notation, limits and primary source notes

Source links support library capabilities and statutory context; the architecture and trade-offs are design recommendations.

UML NOTATION

Filled diamond: owned composition. Solid association: structural relationship. Dashed dependency: uses/creates. Dashed line with hollow triangle: interface realization. Multiplicities indicate cardinality. Sequence alt/loop blocks describe alternatives/repetition; asynchronous arrows and job-state reads do not imply shared object ownership. State diagrams describe lifecycle rather than payment balance.

VALIDATION STATUS

All 28 PlantUML source files are rendered and checked for syntax errors. The PDF pages are visually reviewed for clipping, unreadable overlaps and consistency. The design has been reviewed for dependency direction, template history, transaction boundaries and failure handling. Application code has not yet been implemented, so no claim of executed application tests, measured performance or complete GST compliance is made.

PRIMARY SOURCES

Qt Designer - https://doc.qt.io/qtforpython-6/tools/pyside-designer.html

Qt threads and queued signals - https://doc.qt.io/qtforpython-6.10/overviews/qtdoc-threads-qobject.html

ReportLab tables and pagination - https://docs.reportlab.com/reportlab/userguide/ch7_tables/

docxtpl templates, images and escaping - https://docxtpl.readthedocs.io/en/latest/

SQLAlchemy SQLite behaviour - https://docs.sqlalchemy.org/en/20/dialects/sqlite.html

SQLite transactions - https://www.sqlite.org/lang_transaction.html

SQLite backup API - https://www.sqlite.org/backup.html

Import-linter forbidden contracts - https://import-linter.readthedocs.io/en/latest/contract_types/forbidden/

GST Rule 46: invoice numbering and fields - https://taxinformation.cbic.gov.in/content-page/explore-rules/1000136/1000001

GST Rule 56: electronic edit/deletion log - https://taxinformation.cbic.gov.in/content-page/explore-rules/1000149/1000001